

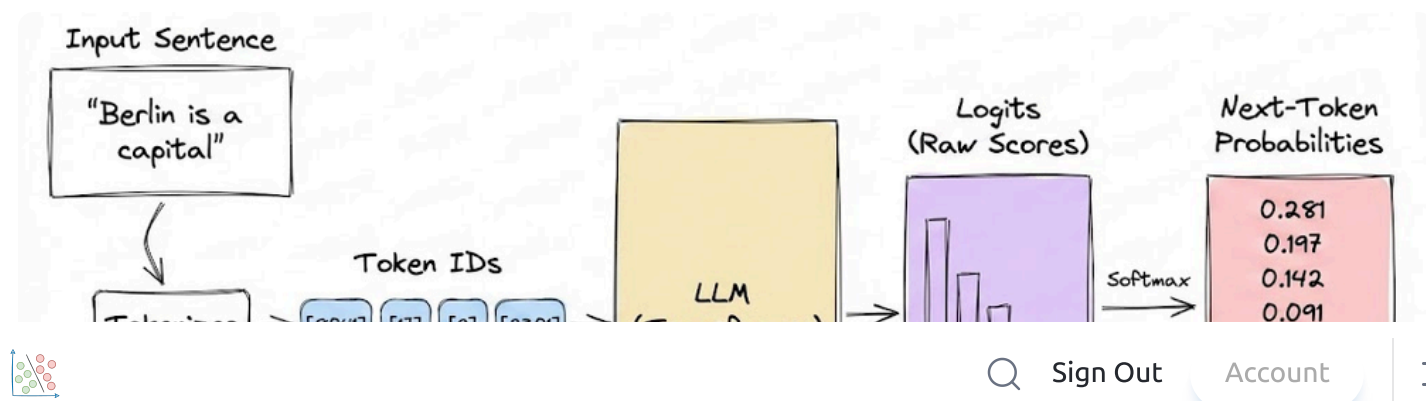
27 min read

Context Engineering: Foundations, Categories, and Techniques of Prompt Engineering

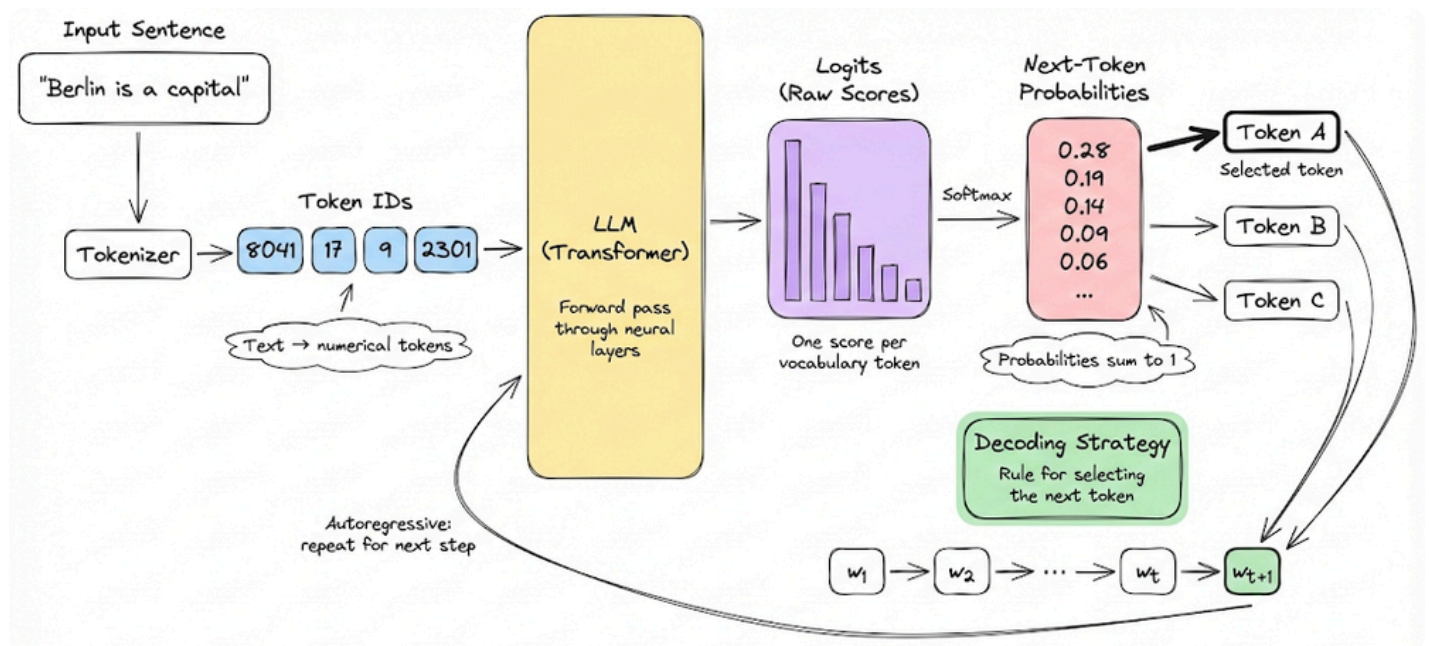
LLMOps Part 5: An introduction to prompt engineering (a subset of context engineering), covering prompt types, the prompt development workflow, and key techniques in the field.

Recap

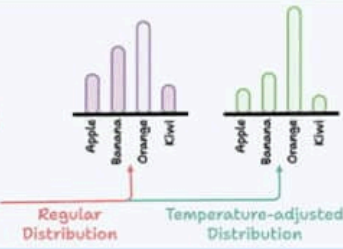
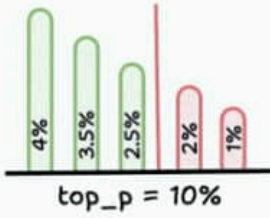
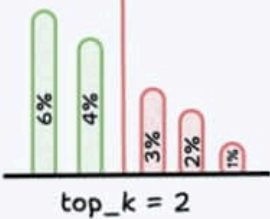
In Part 4, we explored key decoding strategies, sampling parameters, and the general lifecycle of LLM-based applications.



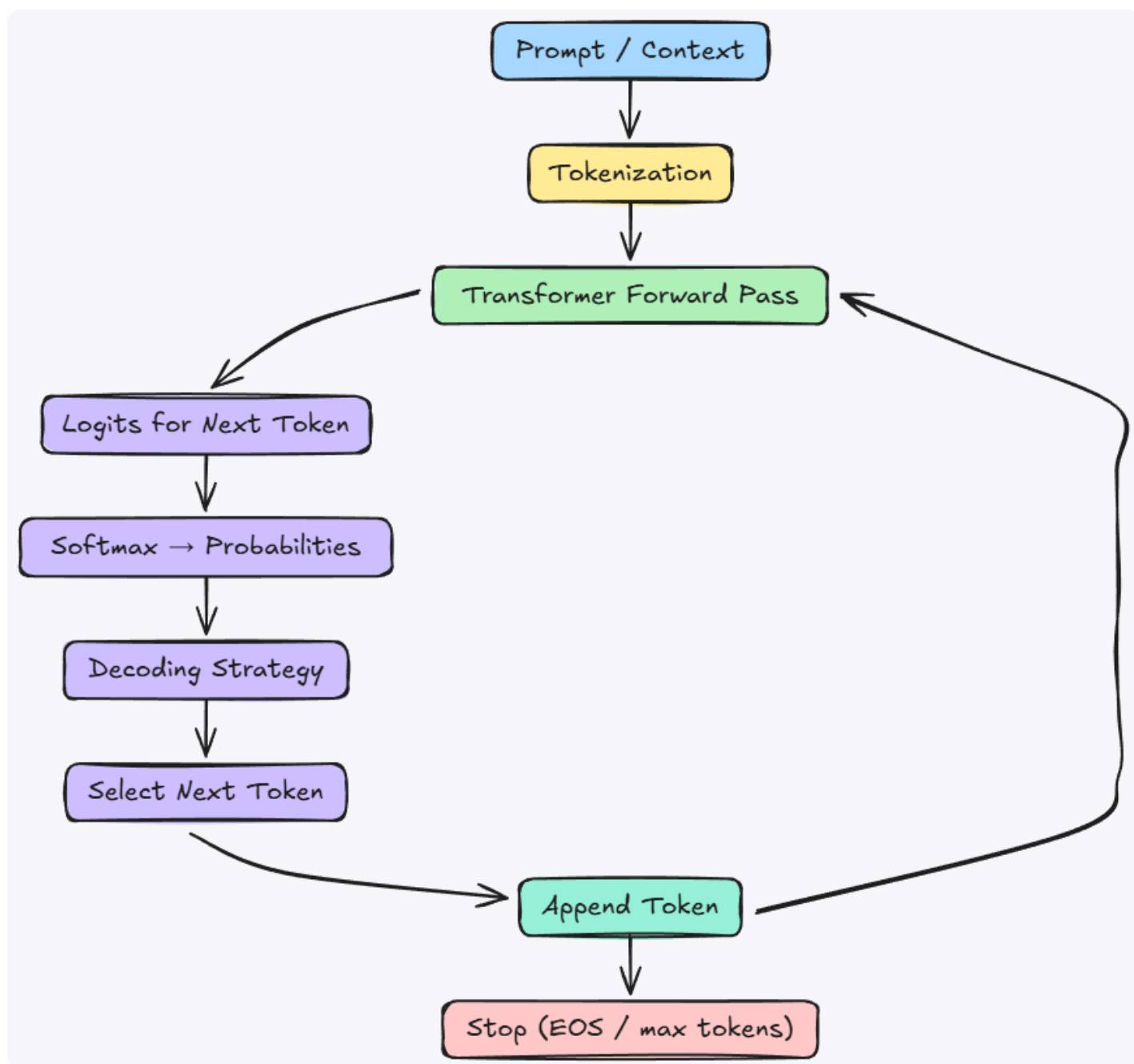
We began by introducing the concept of decoding, and how it functions. We explored the key decoding strategies in detail: greedy decoding, beam search, top-K, top-P, and min-P strategies.



We then went ahead and understood the some of the major generation parameters, like temperature, top-P, top-K, max tokens, etc. in detail.

Max tokens	 The sun sets, painting the sky in fiery hues of orange and pink. A gentle breeze whispers through the trees. Max = 15 Token count 15 Upper limit for the number of tokens the model generates. Value Range = 1 to infinity
Temperature	  Controls randomness in output. A higher temperature makes it more creative and diverse. Value Range = 0 to 2 (common range)
Top_p	  Controls probability distribution that is considered when sampling tokens. Value Range = 0 to 1
Top_k	  Limits the number of top probable tokens to sample from. Value Range = 1 to infinity

We also examined the complete text generation workflow and sketched a mental map of how LLMs function.



Moving ahead, we grounded the learned concepts about decoding and generation parameters with hands-on experiments. We compared the greedy and beam search decoding strategies, and showed how they operate at the token level.


```

=====
AUTOREGRESSIVE GENERATION (GREEDY)
=====
Prompt: "The cat sat on the"

=====
STEP 1
=====
Current text: "The cat sat on the"

Top-n next-token predictions:
Rank      Token Probability
  1      ' mat'      0.8426
  2      ' cat'      0.0247
  3      ' fence'    0.0110
  4        '\n'      0.0094
  5      ' window'   0.0046

Selected token: ' mat'

```

Greedy decoding

```

=====
AUTOREGRESSIVE GENERATION (BEAM SEARCH)
=====
Prompt: "The cat sat on the"
num_beams: 3, length_penalty: 1.0

===== STEP 1 =====

Beam 1 prefix:
The cat sat on the
+ ' mat' (logP=-0.1712)
+ ' cat' (logP=-3.6994)
+ ' fence' (logP=-4.5077)

Selected beams:
[1] score=-0.1712 | The cat sat on the mat
[2] score=-3.6994 | The cat sat on the cat
[3] score=-4.5077 | The cat sat on the fence

```

Beam search

We also compared different values for generation parameters (temperature, top-P, top-K), clearly demonstrating the influence the model's responses.

EXPERIMENT 1: TEMPERATURE COMPARISON

PROMPT: 'The future of artificial intelligence is'

Temperature = 0.01 (Very low randomness):

Output 1: in the hands of the people. The future of artificial intelligence is in the hands of the people. The future of artificial intelligence is in the hands of the

Temperature = 0.7 (Balanced):

Output 1: here. And it is one that will change the way we work and interact with the world. AI is changing every industry, and you can be one of them. Here are five way

Temperature = 1.5 (High Creativity):

Output 1: in the process of coming .

In some sense the technology in question dates from prehistory -- in terms of historical progress -- what we would perhaps today be most concerned about is how society

Temperature comparison

EXPERIMENT 2: TOP-P (NUCLEUS SAMPLING) COMPARISON

PROMPT: 'Once upon a time in a distant galaxy'

Top-p = 0.5 (Conservative):

Output 1: , there was a group of aliens who lived in a world called "The Delta." They had a unique way of communicating with each other. The aliens used a special

Top-p = 0.9 (Balanced):

Output 1: , there lived a very wise and powerful man named Zephyr. He was a great leader, but also a great prankster. Zephyr loved to create jokes and always found

Top-p = 1.0 (Full Distribution):

Output 1: , there lived an old wise owl named Quagga. Quagga had a very weird hobby: he loved solving puzzles and puzzles made him smarter and wiser. He was always

Top-P comparison

EXPERIMENT 3: TOP-K COMPARISON

PROMPT: 'The recipe for chocolate chip cookies requires'

Top-k = 10 (Very Restrictive):

Output 1: 2 cups of flour and x cups of sugar. Rick decides to make a smaller batch of cookies using 1 cup of flour and 2 cups of sugar. How many cups of sugar
If

Top-k = 50 (Balanced):

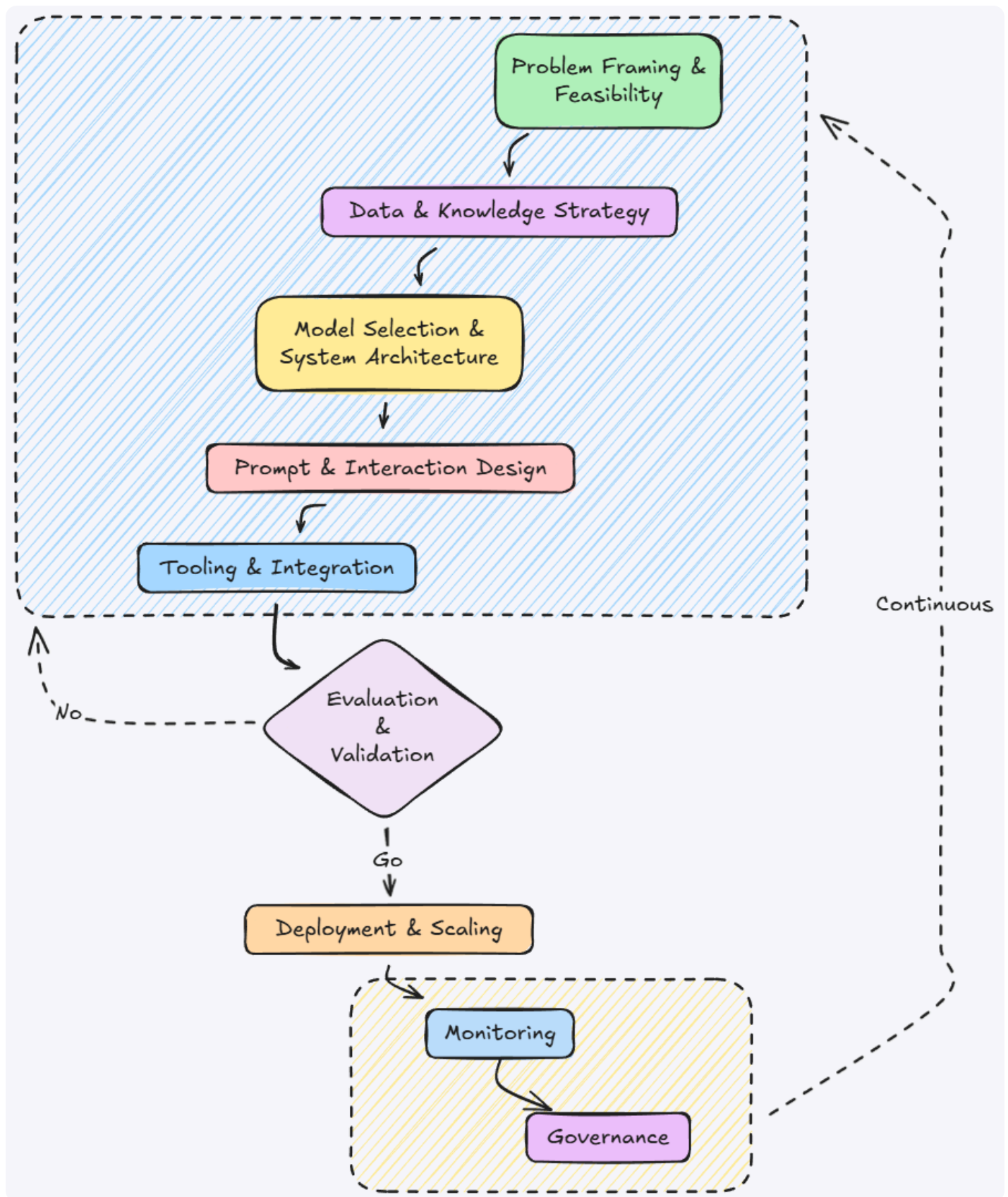
Output 1: 1.5 cups of flour, 0.75 cups of sugar, and 0.5 cup of butter. If a baker only has 0.35 cup of flour and 0.2 cups of sugar, how many

Top-k = 100 (Permissive):

Output 1: 2 cups of flour, 1 1/4 cups of sugar, and 1 1/2 cups of butter. If a baker only has 2 cups of sugar available, how many cups of flour and butter can be

Top-K comparison

Finally, we transitioned towards more of LLMOps by exploring the general lifecycle of LLM-based applications and its different stages.



By the end of Part 4, we had developed a clear understanding of the core concepts and components underlying LLMs, along with a strong LLMOps mindset gained through

understanding the application lifecycle.

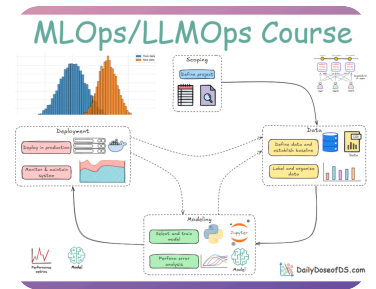
Although this chapter is not strongly related to the previous part, we strongly recommend reviewing Part 4 first, as it helps maintain the natural learning flow of the series.

Read it here:

Building Blocks of LLMs: Decoding, Generation Parameters, and the LLM Application Lifecycle

LLMOps Part 4: An exploration of key decoding strategies, sampling parameters, and the general lifecycle of LLM-based applications.

 Daily Dose of Data Science • Avi Chawla



In this and the next few chapters, we will be exploring context (and prompt) engineering. This chapter, in particular, focuses on the fundamentals of prompt engineering.

As always, every notion will be explained through clear examples and walkthroughs to develop a solid understanding.

Let's begin!

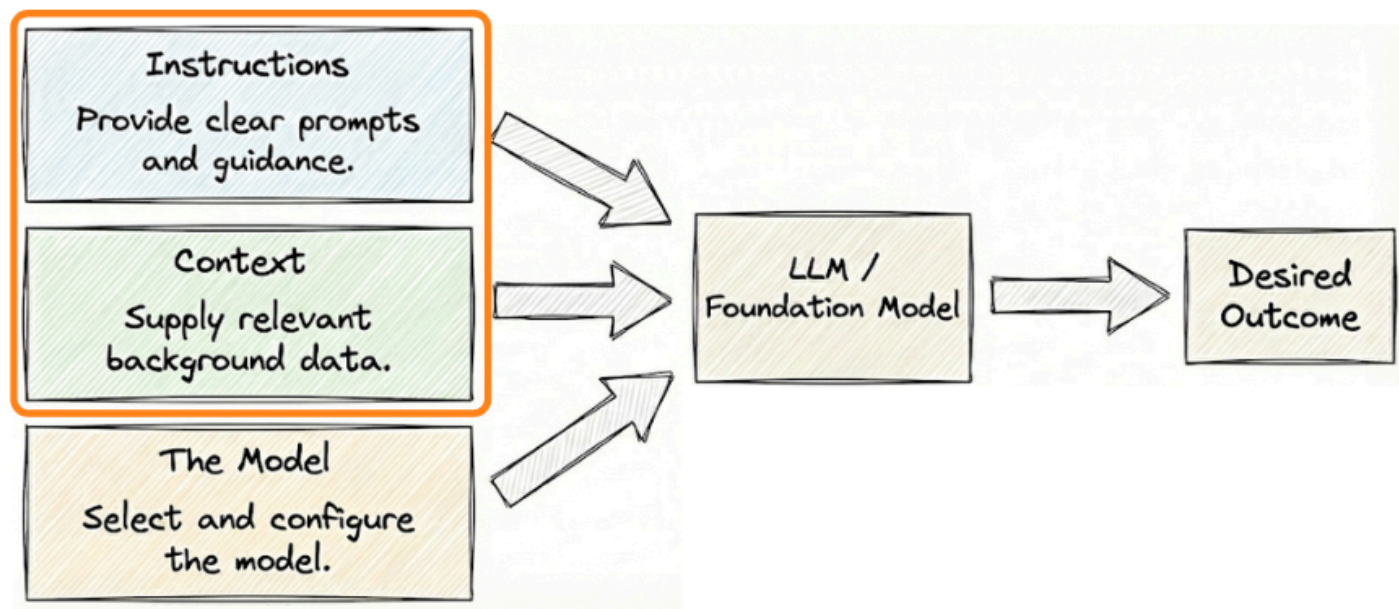
Introduction

Large Language Models (LLMs) have unlocked powerful new capabilities in software, but harnessing them in production systems (LLMOps) requires more than just plugging in an API.

Two critical disciplines have emerged in recent years:

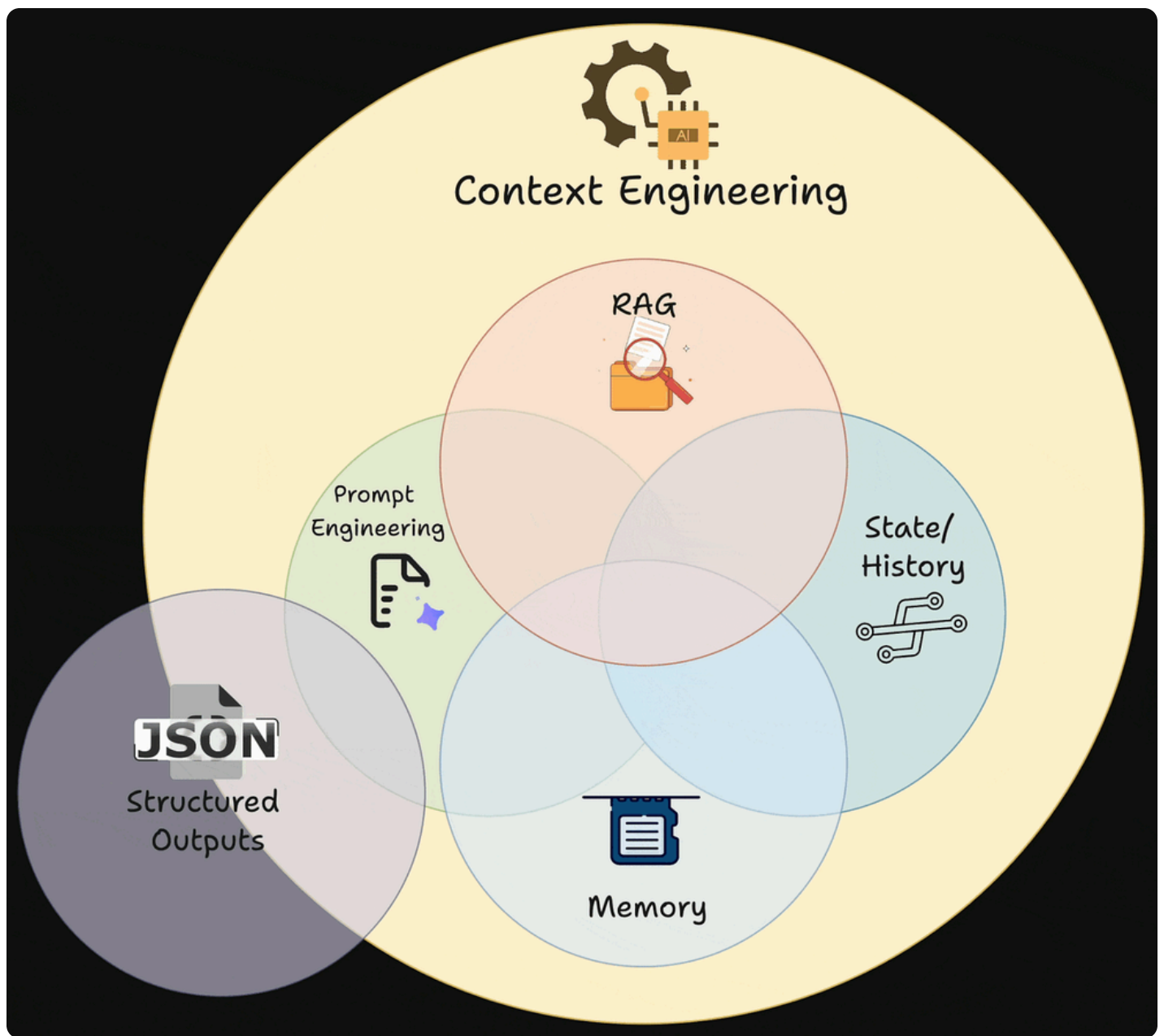
- Prompt engineering
- Context engineering

Prompt engineering focuses on how we design the textual inputs (prompts) to guide an LLM's behavior, while context engineering is about managing the entire information flow, structuring the surrounding data, tools, and environment that feed into the model. Together, these techniques enable us to build reliable and efficient LLM-powered applications.



[Prompts and context are two essential levers of AI engineering](#)

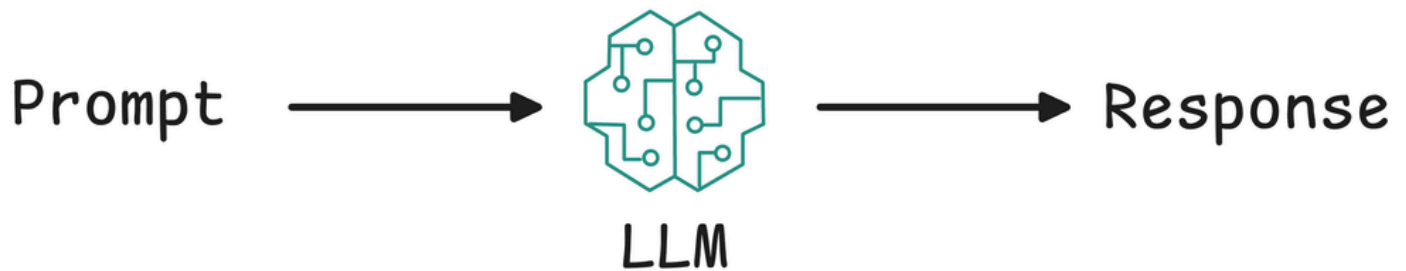
👉 Just a heads up: Prompt engineering should not be seen as unrelated to context engineering. Prompt engineering is a subset of context engineering, since prompts are one way (to a certain extent) of shaping the model's context. We mentioned prompt engineering separately because it is a large and mature discipline in its own and has been widely practiced for much longer, even before context engineering was popularized as a unified concept.



Now let's go ahead and explore the core concepts of prompt engineering, a key subdomain of context engineering. We will begin with the fundamental principles, followed by a practical workflow for developing effective prompts. Finally, we will examine various prompt types and advanced prompting techniques.

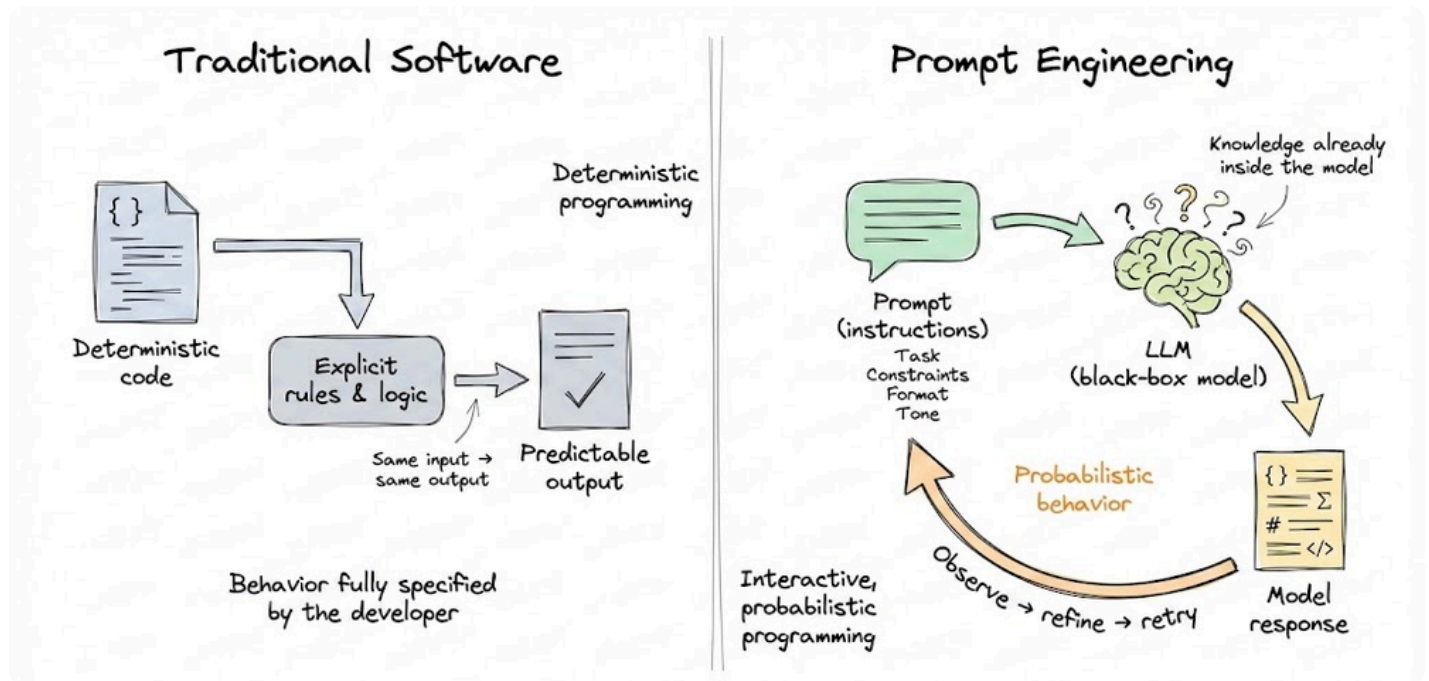
Fundamentals of prompting

A prompt is the input given to an LLM to draw out a response, it can be a question, instruction, or any text the model completes or responds to.



From an engineering perspective, a prompt is not just casual text; it is a piece of logic we design to program the model's behavior (often called “soft programming”).

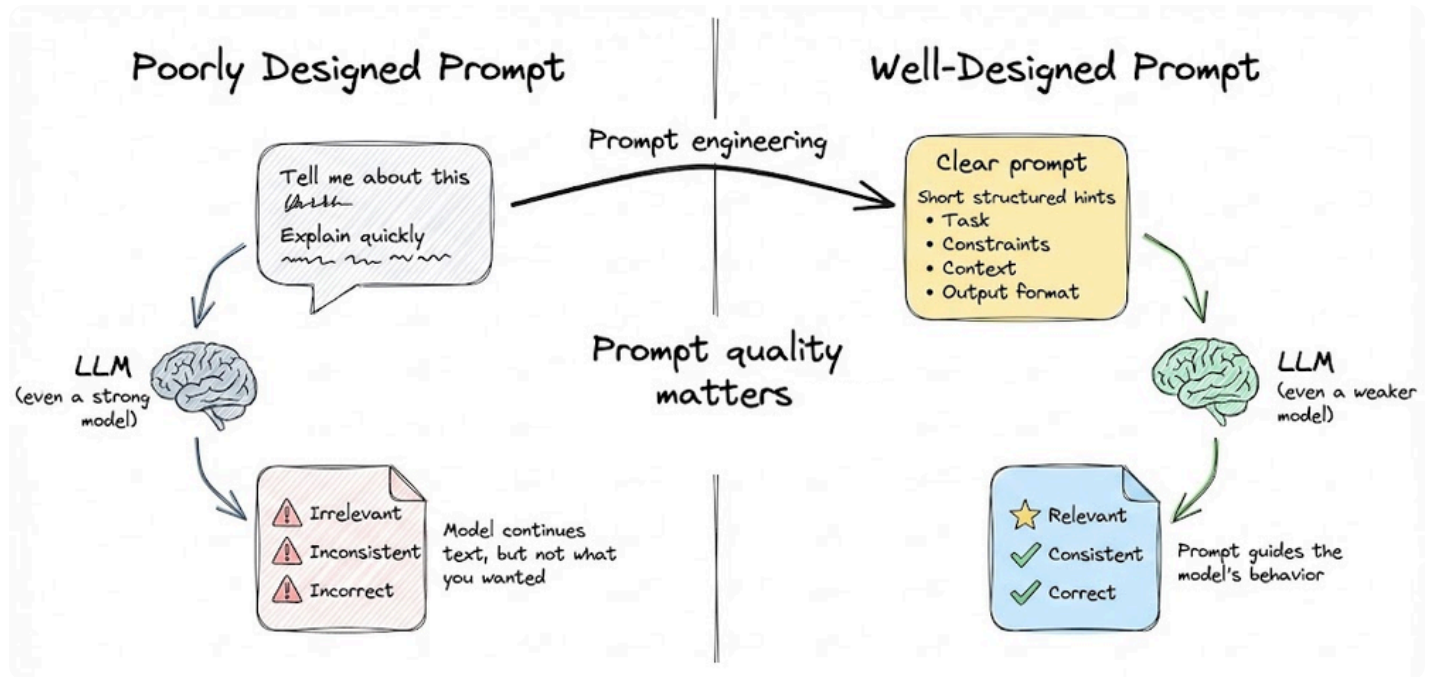
In essence, prompt engineering means writing instructions that effectively leverage the knowledge already present in the model. Unlike traditional software where we write deterministic code, prompting is more of an interactive, probabilistic programming: we guide a black-box model with instructions, then observe how it behaves, and refine our approach.



Why is prompt engineering needed?

LLMs are trained to continue text in a plausible way, but to get the specific output we want, we must craft the input skillfully. A poorly designed prompt can lead to irrelevant, inconsistent or incorrect outputs, even from a very capable model.

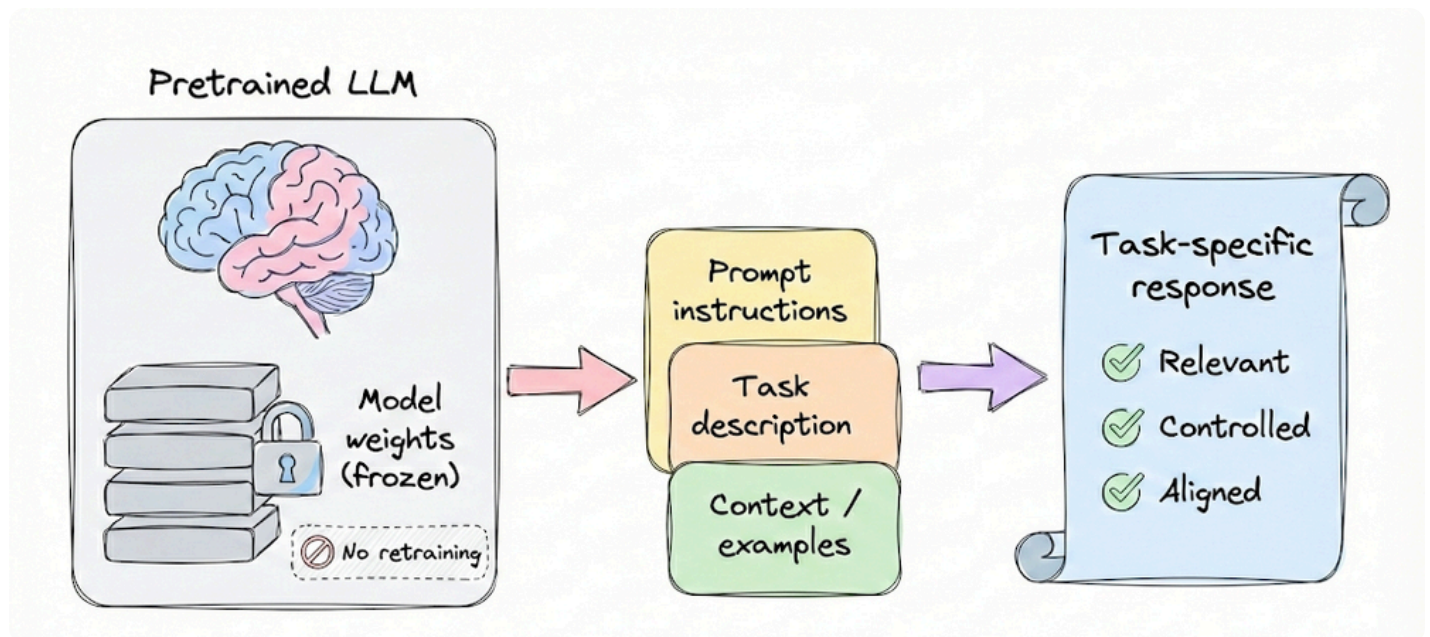
On the other hand, a well-designed prompt can make even a weaker model perform surprisingly well on a given task. Prompt engineering has therefore become “the skill of designing prompts that guide a generative AI model toward the kind of response you actually want”.



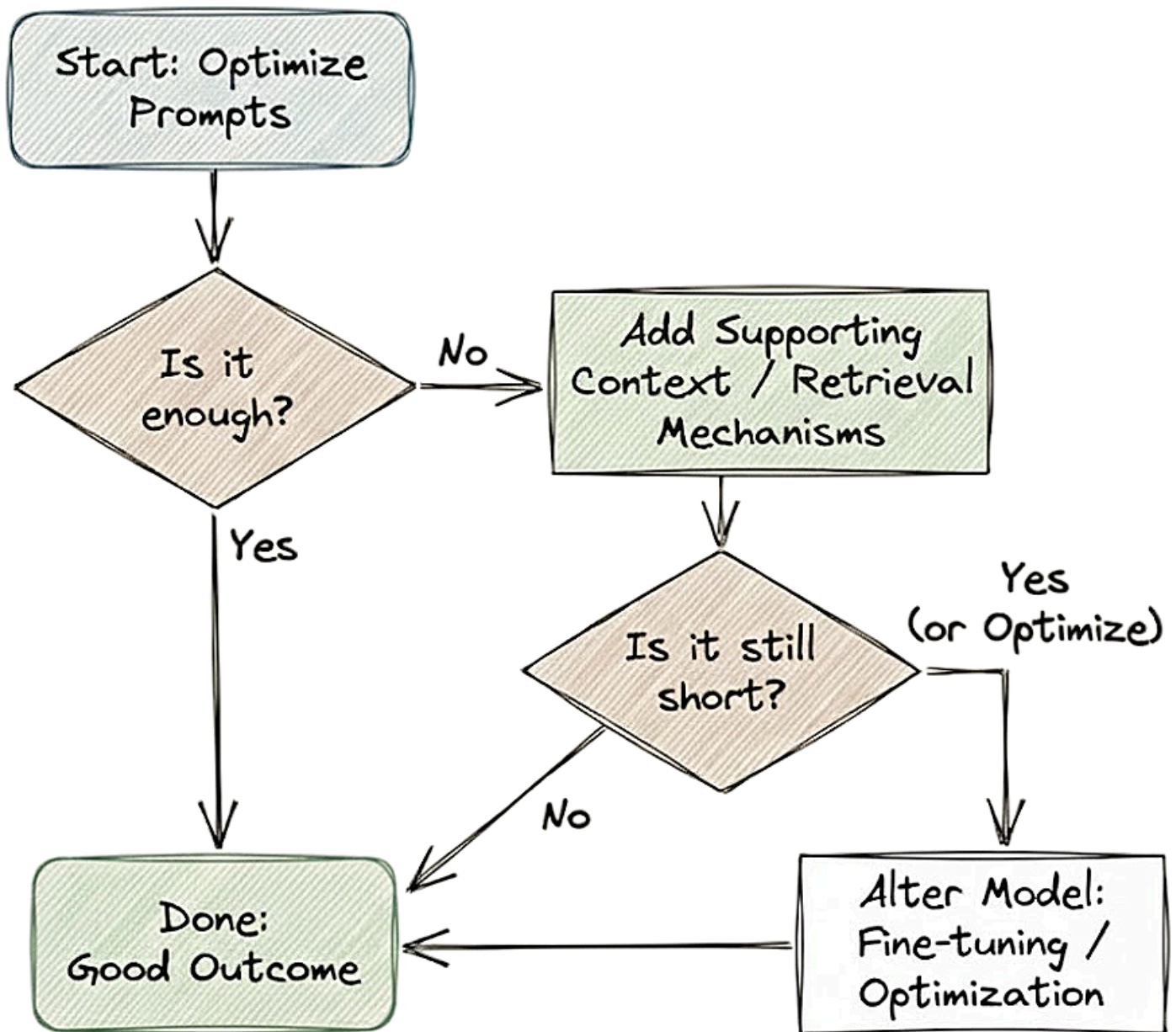
The system perspective

From a system standpoint, prompting is a first-class component of the application. We treat prompts like code: we design them methodically, test them, version-control them, and continuously improve them.

Importantly, prompt engineering is often the most accessible way to adapt an LLM to your needs without model retraining. It requires no updates to the model's weights, instead, you leverage the model's already present knowledge by providing instructions and context.

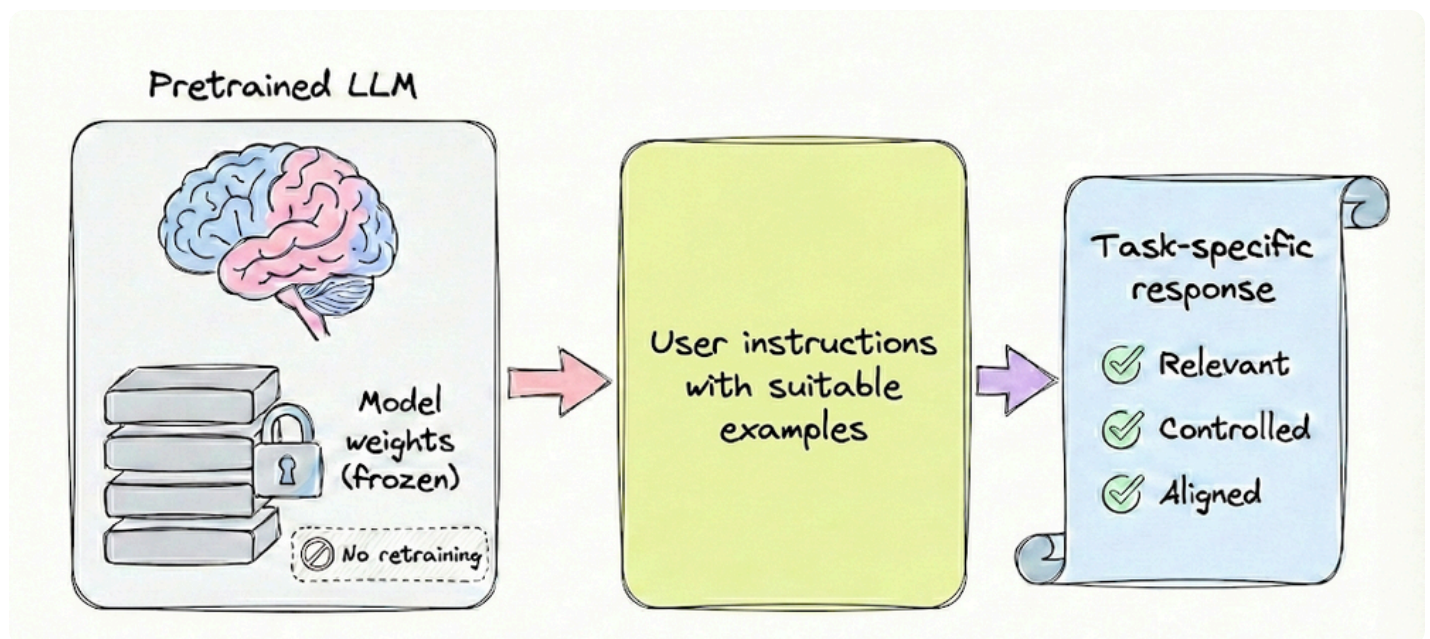


This makes prompt engineering fast to iterate on and deploy, which is why many early LLM applications relied solely on prompting. However, it's not a silver bullet, more complex tasks often require going beyond skillful prompting to exploring a broader context pipeline, fine-tuning, or additional data.



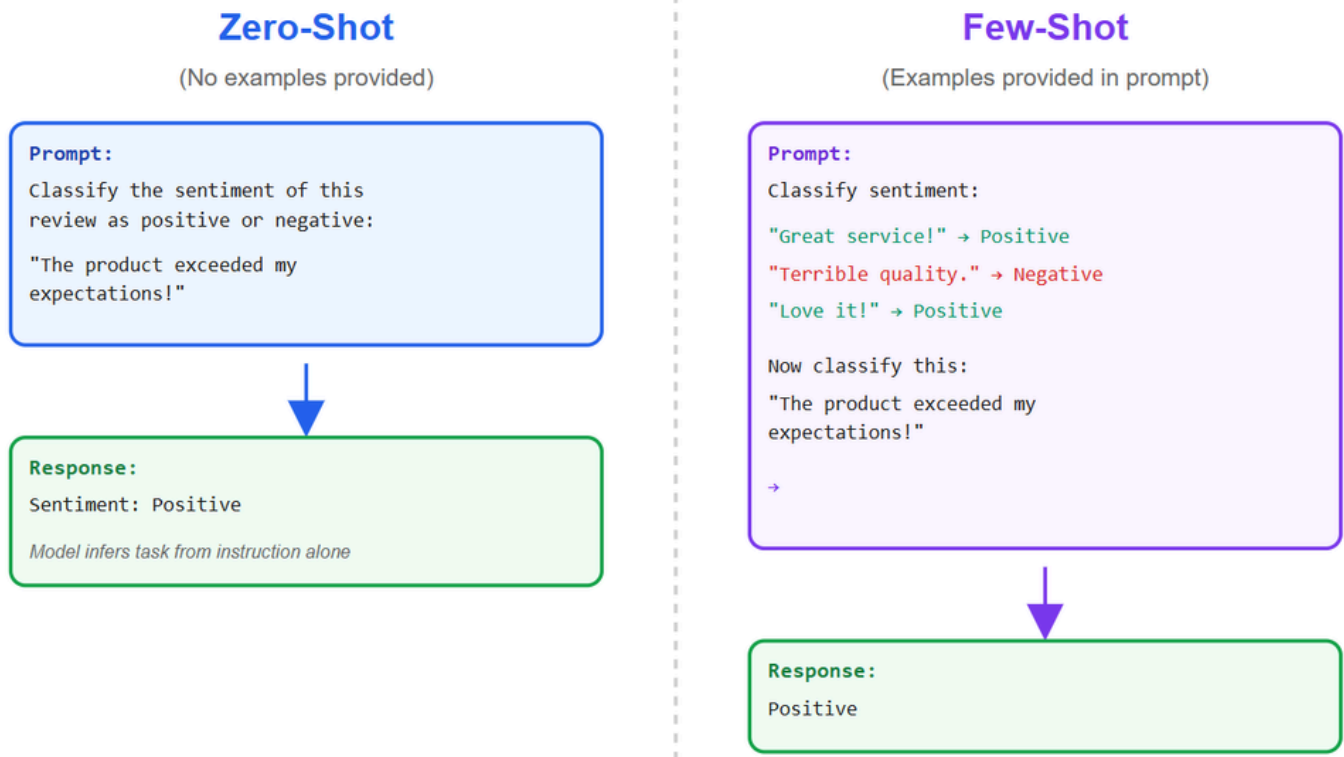
In-context learning

Prompting is closely tied to the concept of in-context learning. Researchers discovered with GPT-3 that large models can learn tasks from examples given in the prompt itself, without any parameter updates.



For instance, if you prompt the model with a few example question-answer pairs (few-shot prompting), the model can infer the pattern and apply it to a new question. If you provide no examples, expecting the model to solve the task from just instructions, that's zero-shot prompting.

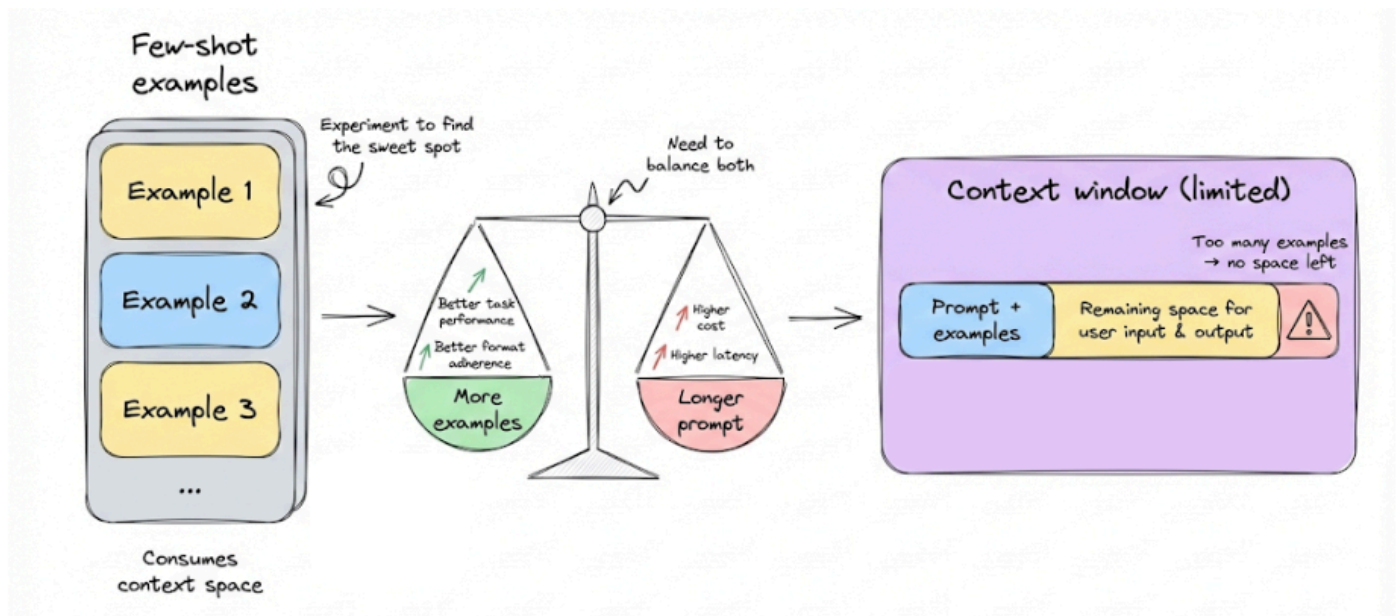
Zero-Shot vs Few-Shot Prompting



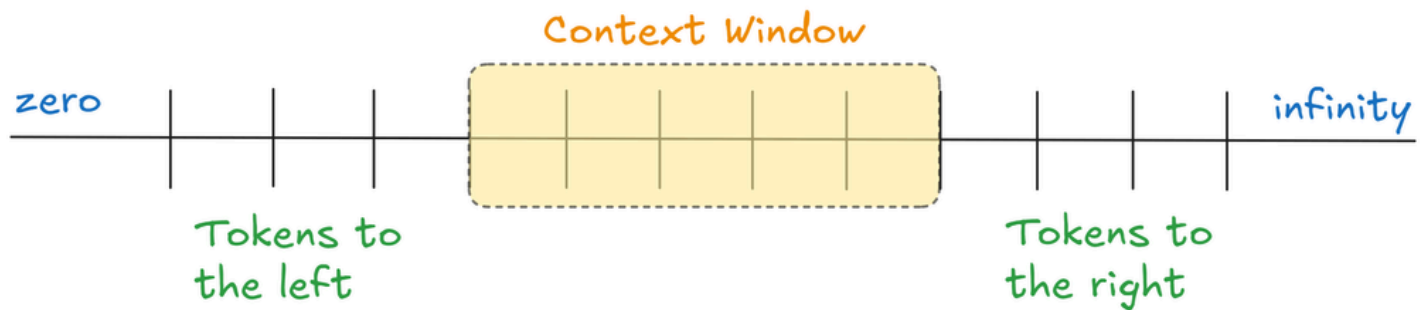
Few-shot prompting (sometimes called few-shot in-context learning) often improves accuracy by showing the model what format or style of answer is expected. For example, adding 5 examples makes it a “5-shot” prompt. GPT-3’s paper “Language Models are Few-Shot Learners” highlighted this ability.

However, there are diminishing returns with newer advanced models: experiments have shown GPT-4 sometimes doesn’t gain much from few-shot examples on certain tasks compared to zero-shot. This is likely because newer models are better at following instructions out-of-the-box, so a clear zero-shot instruction often suffices for them.

But in niche domains, a few examples can still boost performance significantly if the model’s training data lacked those patterns. In practice, you should experiment to find the optimal number of examples for your task: balancing performance versus the added prompt length (which increases cost and latency).



Note that few-shot prompts use up context space, so you can't include too many examples given the context window is limited.



In summary, the fundamental idea is that prompts provide both instructions and context to the model. They can include a description of the task, relevant information or data, and examples of the task being performed.

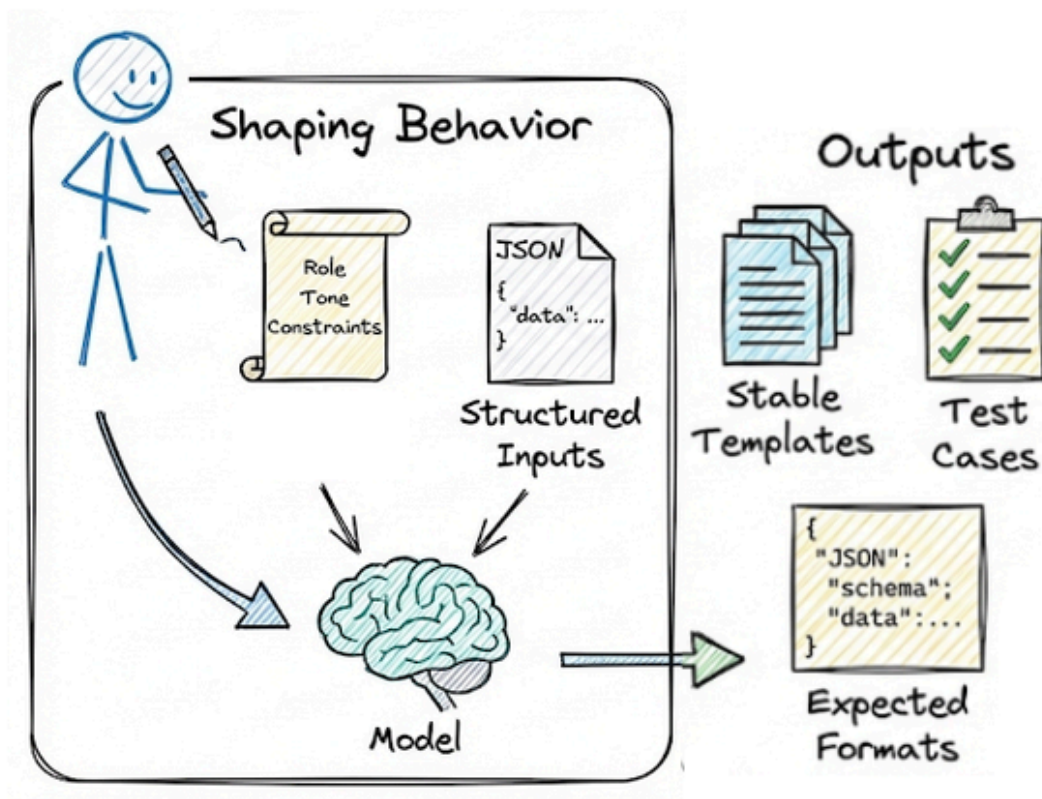
Overall, prompt engineering is fundamentally about constructing these elements in a way that the model reliably produces the desired behavior. We now turn to a systematic approach to developing prompts.

Systematic prompt development workflow

Developing an effective prompt is an iterative engineering process. Rather than guess-and-check ad hoc, it helps to follow a structured workflow:

Define the task and success criteria

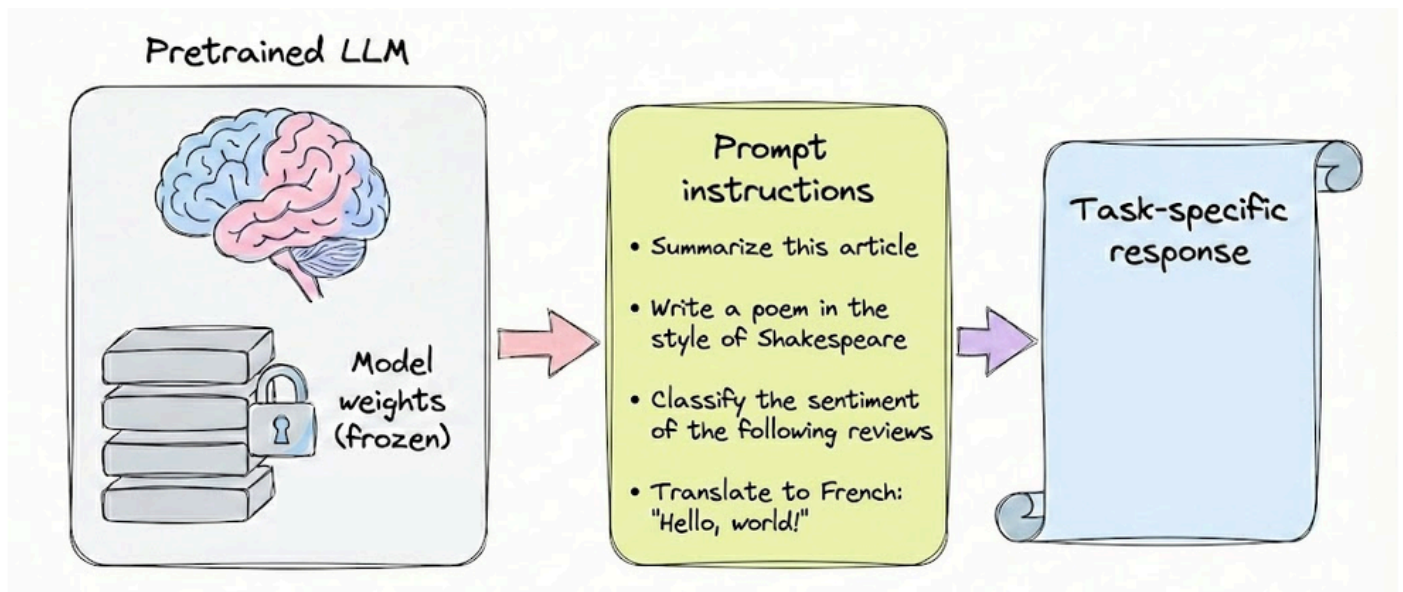
Start by clearly defining what you want the model to do (e.g., “extract the total price from an invoice email and output a JSON”). Identify what a correct output looks like and any constraints (formatting, tone, length, etc.).



This is analogous to writing a spec; it will guide your prompt design and evaluation criteria.

Draft an initial prompt

Create a first version of the prompt. At a minimum, state the instruction clearly. Depending on the task, you might include some examples or a specific format.



Write an initial prompt

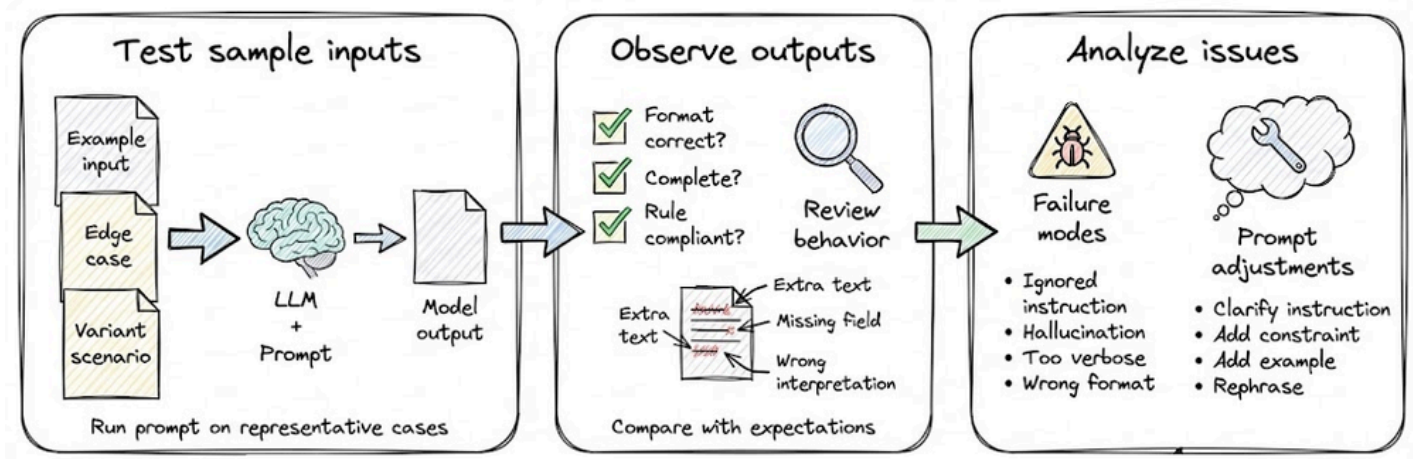
Test sample inputs and analyze

Run the model with your prompt on a variety of test queries or scenarios (via various user prompts or supporting context). Observe the outputs:

- Do they match the expected format?
- Are they correct, complete, and compliant with any rules?
- Any issues? For example, the model misunderstands something or includes extra text.

When the output is not what you want, analyze why.

- Did the model ignore an instruction?
- Did it hallucinate?
- Was it too verbose or not following format?

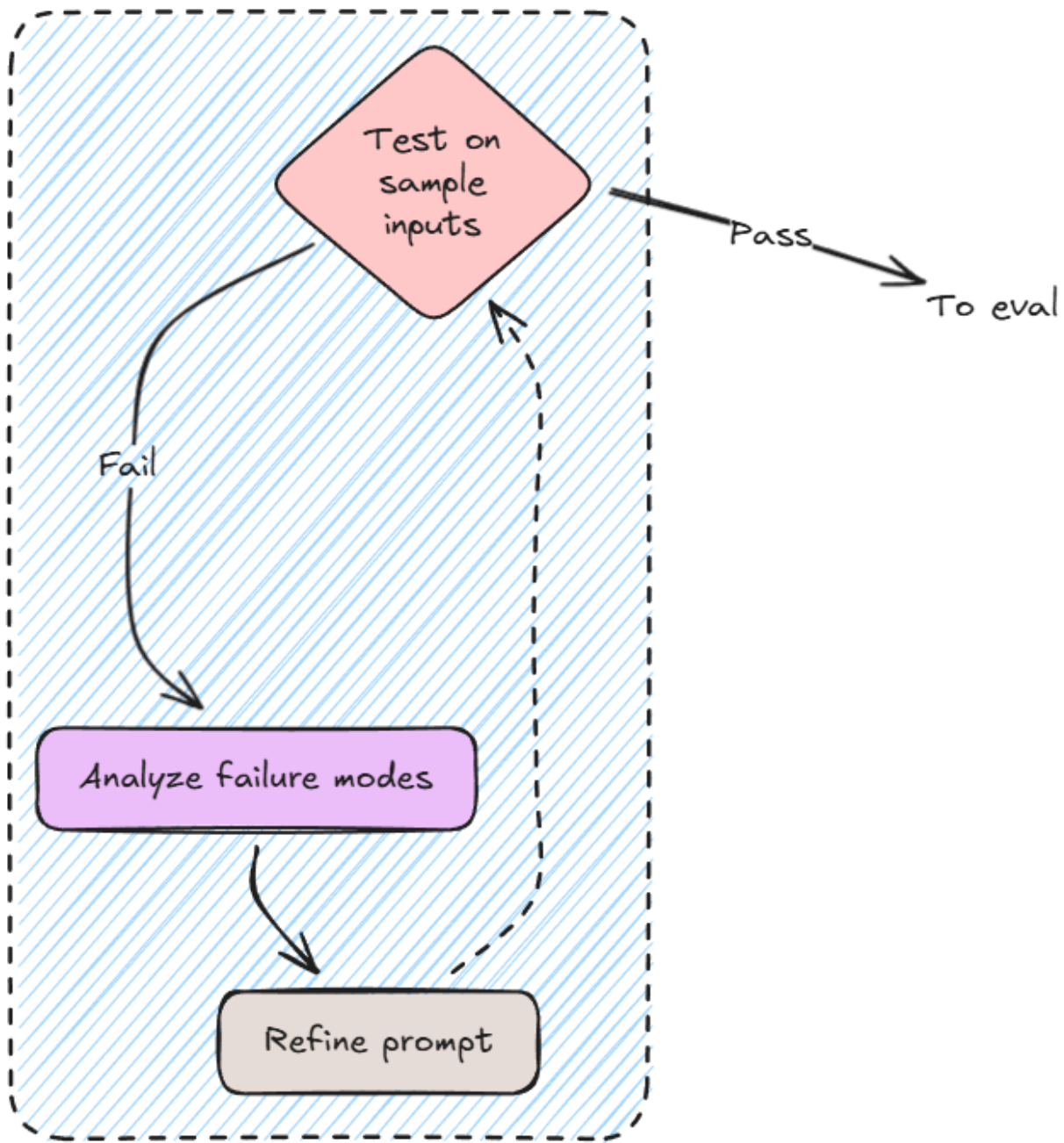


For each issue, think of how you could adjust the prompt to address it. This might mean clarifying the instruction, adding a constraint, giving an example, or rephrasing.

Refine the prompt and iterate

Apply the changes identified. Be as concrete as possible in telling the model what you need. Each refinement is like debugging the prompt.

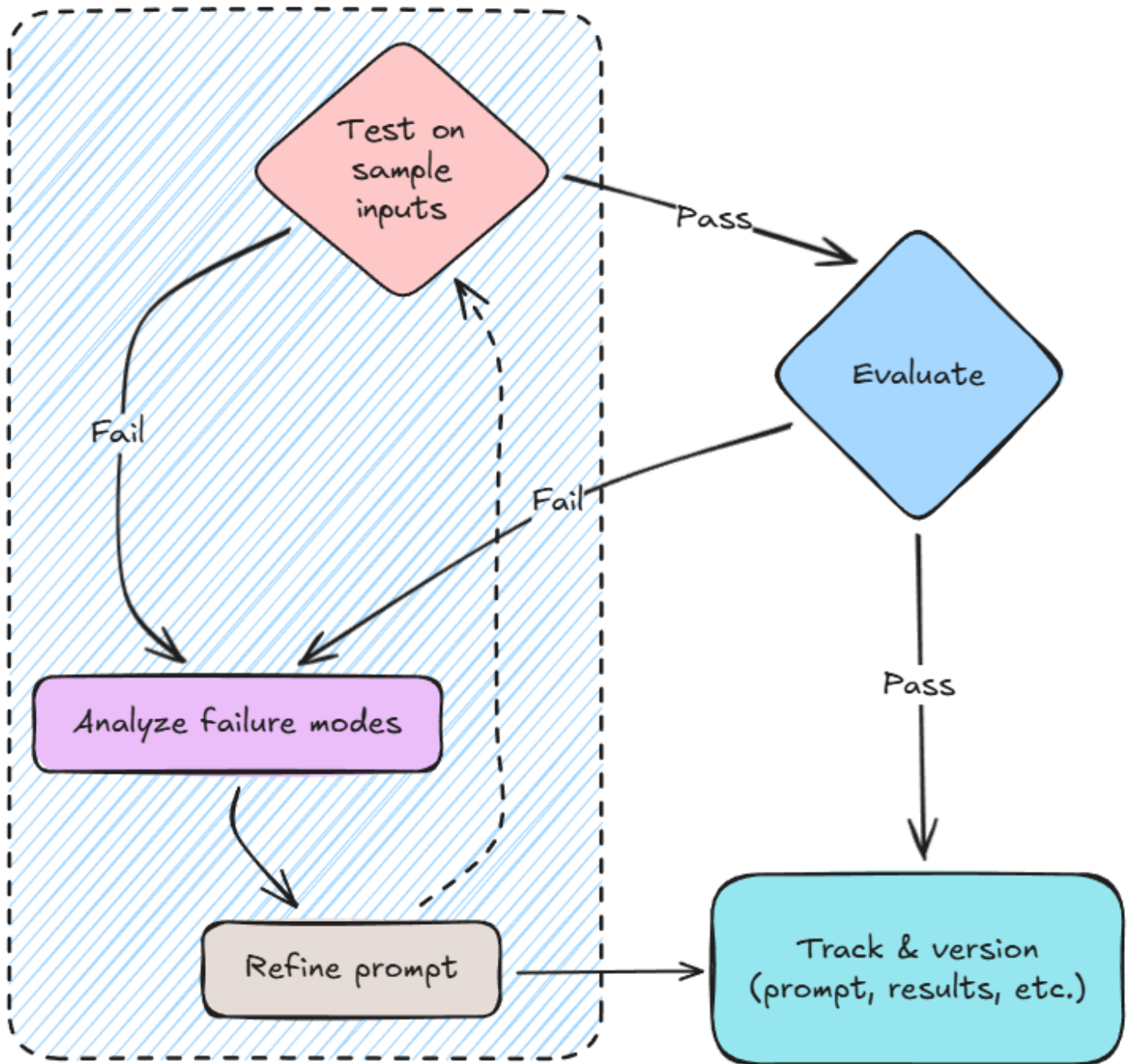
After modifications, iterate again on your examples (and possibly new ones). Compare the new output against the previous one to ensure your changes helped rather than hurt.



This iterative loop continues until the prompt performance is satisfactory. It's common to go through multiple iterations, hence prompt engineering is often described as an experimental, empirical process.

Validation and evaluation

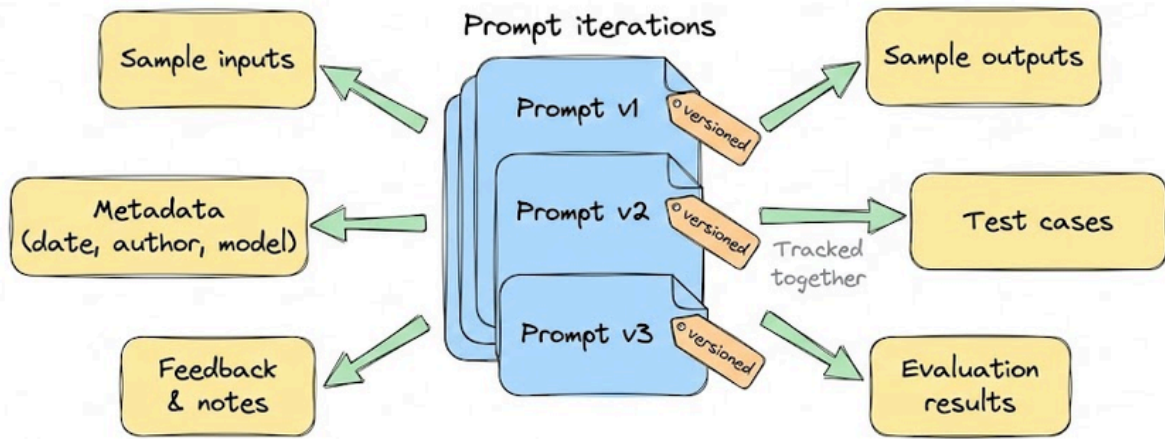
Once the prompt seems good on your dev tests, validate it more rigorously. This might mean using a larger evaluation set or even using automated eval tools.



We'll be diving deep into evaluation in a later chapter, but essentially you want to assess that your prompt achieves the task consistently. If the app is high-stakes, consider a human review of outputs as well.

Tracking and versioning

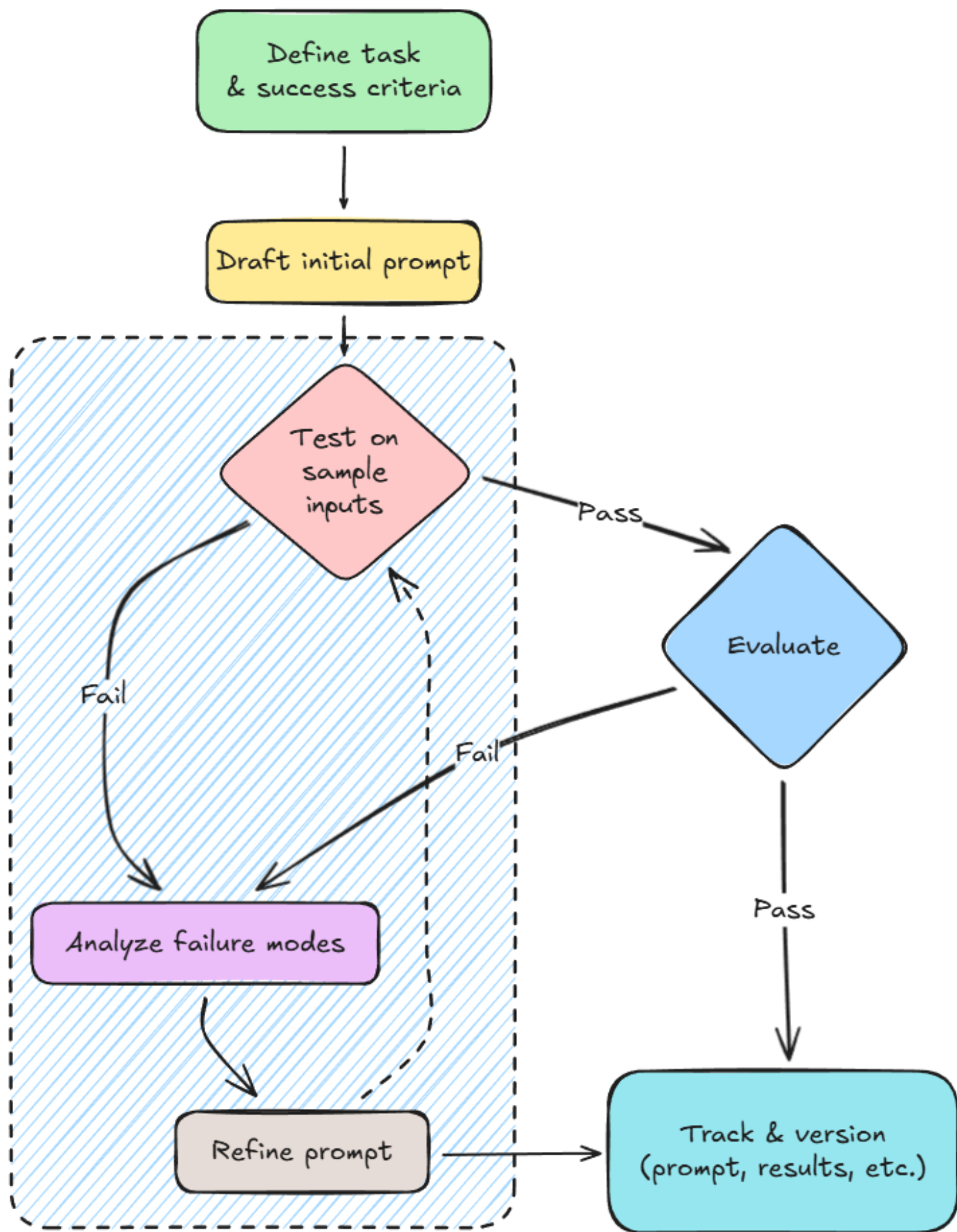
Throughout the process, it is important that we continuously track and version at least the various prompts (initial and refinements) and metadata. Beyond that depending on complexity and stake we could also attach and version associated outputs, testing data, reasons, etc.



This helps us with maintenance of our procedure and experimentation. We'll be discussing more on prompt versioning in the next chapter.

This workflow resembles the traditional software development objectives, applied to prompt design. In fact, as teams scale up prompt engineering, they find that managing prompts requires discipline: without version control and systematic testing, prompt changes can break functionality unexpectedly.

Here is the complete workflow summarized in form of a diagram:



Now, let's examine the types of prompts and techniques we have at our disposal when crafting prompts.

Types of prompts

Not all prompts are alike. Depending on the role and content of the text we include, we can distinguish several prompt types that serve different purposes in an LLM-powered system:

System prompts

A system prompt (or system message) is an initial instruction that sets the overall behavior of the model. It often defines the model's role, tone, or rules that persist throughout the interaction.

Think of it as the “persona and policy” for the AI. For example:

“You are a helpful financial assistant. You will answer in concise language.”



LLM

You are an
expert financial...

Most chat APIs allow a special system message separate from user input. The system prompt usually comes first in the combined input to the model, which can influence the model to

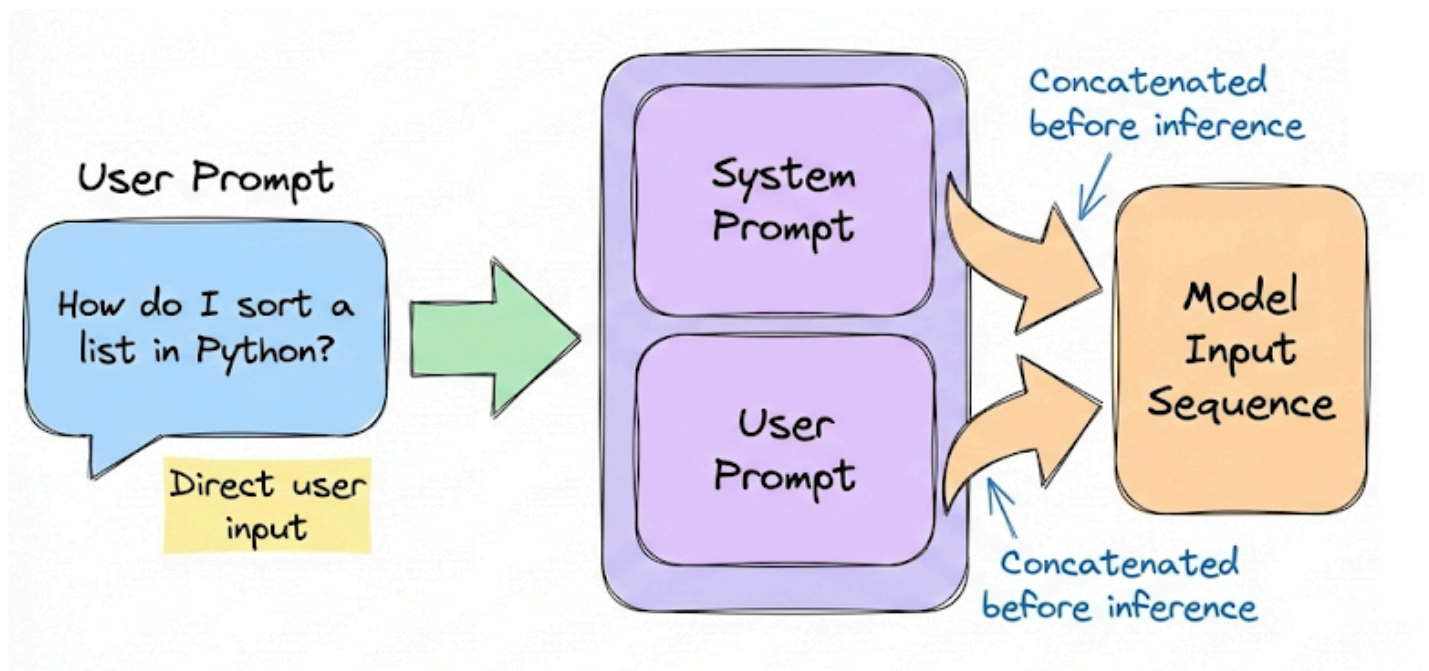
prioritize it.

A well-crafted system prompt can significantly steer the model's behavior and even improve performance on a task. It's also a key tool for mitigation by explicitly instructing the model about do's and don'ts (e.g., "If the user requests disallowed content, refuse."), you establish guardrails.

In summary, use system prompts to set context globally they apply to all user requests in a session.

User prompts

The user prompt is the direct input from the user, e.g., their question or command. In a chat, this is what the user types each turn. From the model's perspective, the system prompt and user prompt are concatenated into one sequence. But conceptually, the user prompt is the immediate task or query the model should address, often phrased as a question or imperative. For instance: "How do I sort a list in Python?"



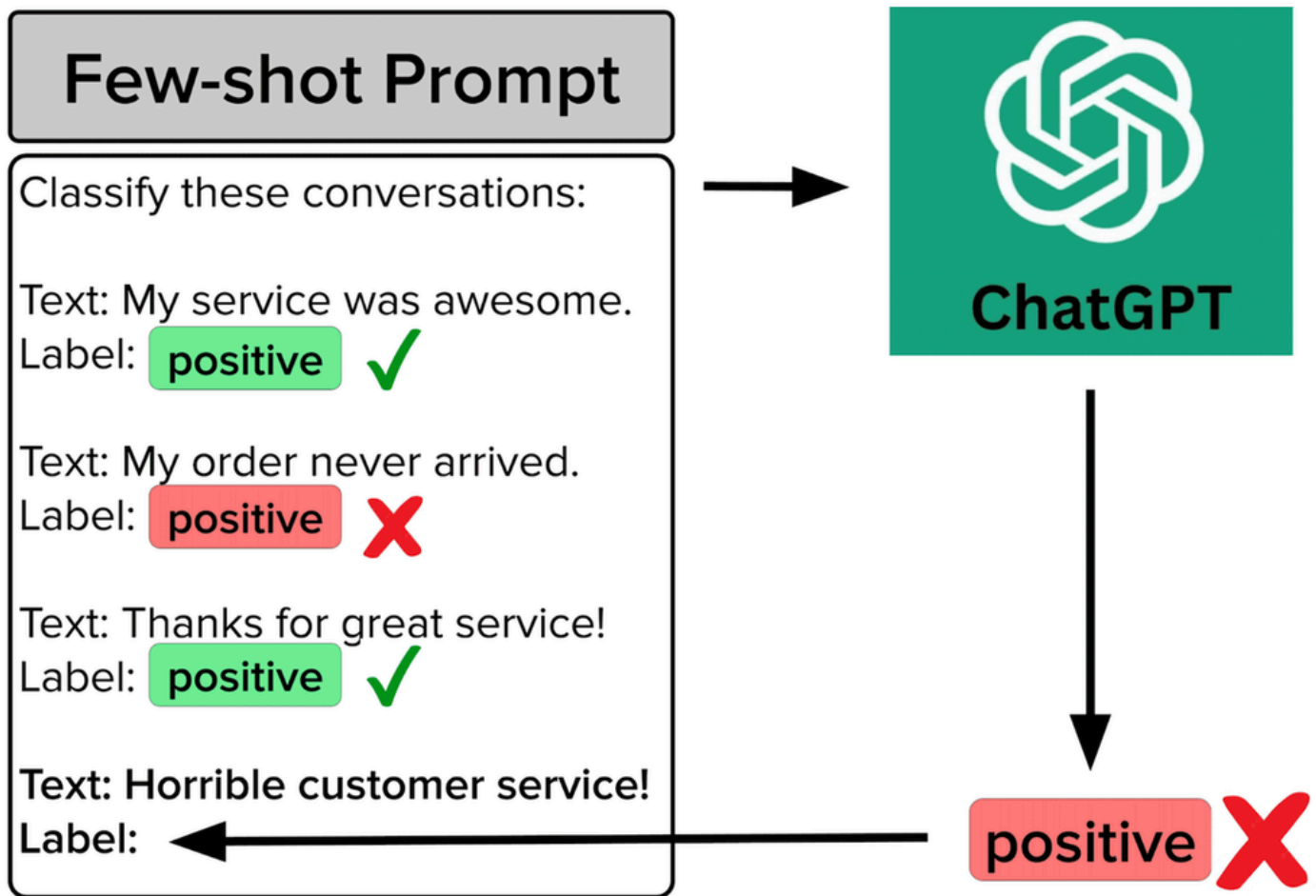
User prompts can be free-form, but as developers we might pre-process or augment them (like reformatting, adding context) before passing to the model. The key is that the user prompt

carries the user's intent. It may be the only prompt content (in simple API usage), or it may work in tandem with a system prompt and few-shot examples.

Few-shot example prompts

These are examples inserted into the prompt to demonstrate the task to the model. They typically consist of a fake user query and a desired model answer, sometimes formatted as Q&A or conversation turns.

Few-shot examples essentially provide context by example, the model sees how tasks should be handled. Few-shot prompts serve as on-the-fly “training” for the model, if we can say that.



As discussed, they can greatly improve performance especially for smaller models or niche tasks, at the cost of consuming context space.

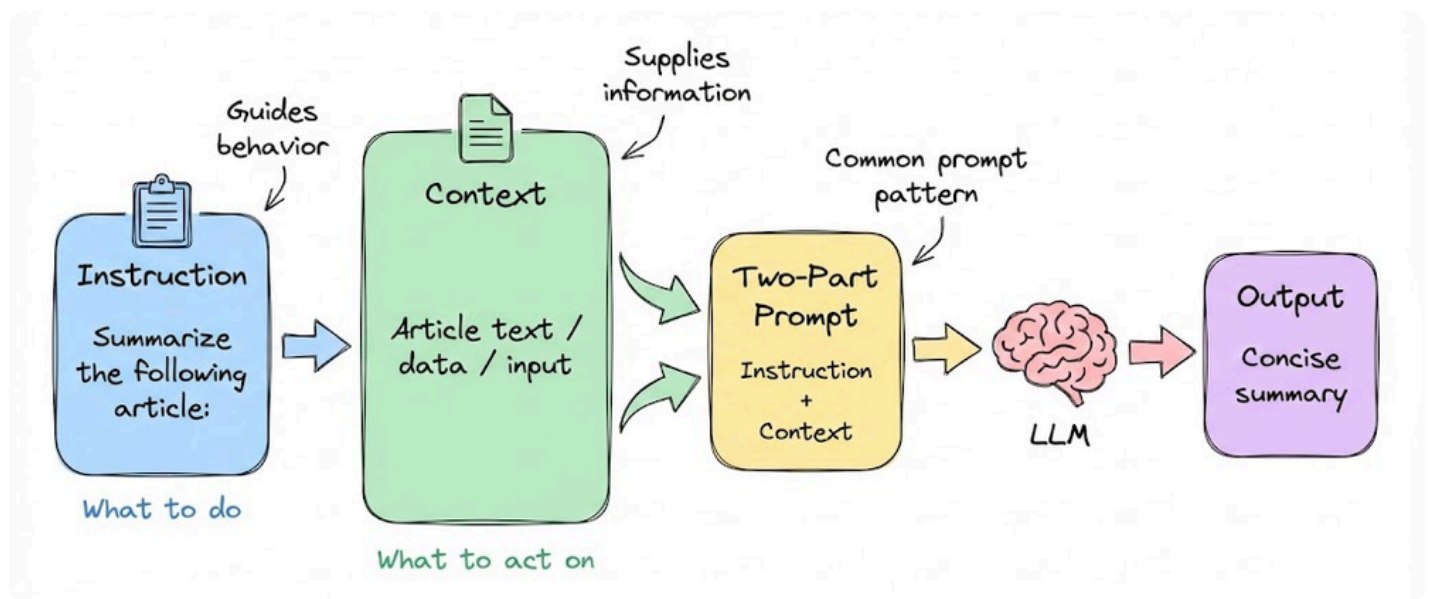
There is an art to selecting good examples. They should be representative of the variety of inputs and edge cases, and ideally, they should be correct and unambiguous (since the model might mimic any mistakes in the examples).

👉 Note that in few shot prompting, ordering can also matter. Sometimes models exhibit recency bias to the last example, so you might need to order by complexity or use randomization during development.

Instruction prompts and contextual prompts

We often distinguish between instructions (telling the model what to do) and context (supplying data to act on). In practice a prompt can include both.

For example, an instruction prompt might say “Summarize the following article:”, and then the article text is included as context. The model then uses that context to fulfill the instruction.



Many prompts follow this pattern of an instruction plus some input data. This could be seen as a “two-part prompt” (instruct, then provide context).

👉 When designing prompts, it’s useful to clearly separate these parts: what is the task (instruction) and what information should be used (context). This clarity helps the model

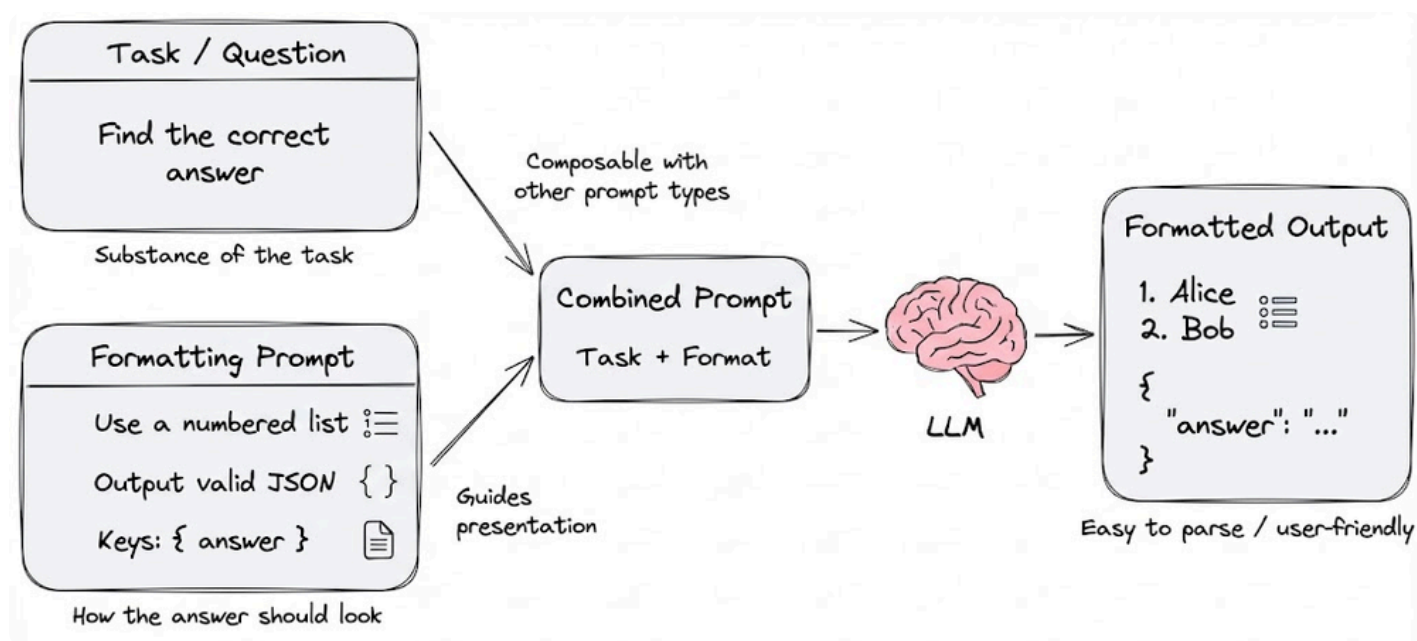
parse your intent correctly.

Formatting prompts

These prompts emphasize the output format or style. They may not change the substance of the answer but how it is presented.

For instance, you might prompt:

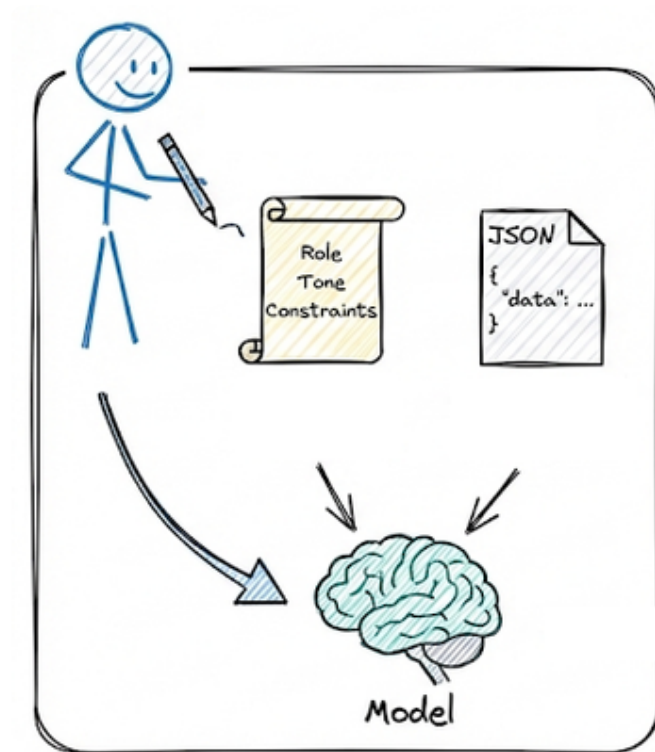
- “List the following names in a numbered list.”
- “Provide the answer in JSON format with keys ‘answer’.”



By specifying a required format, you guide the model to output something that your system can easily parse or that fits user expectations. Formatting instructions can be combined with any of the above types of prompts.

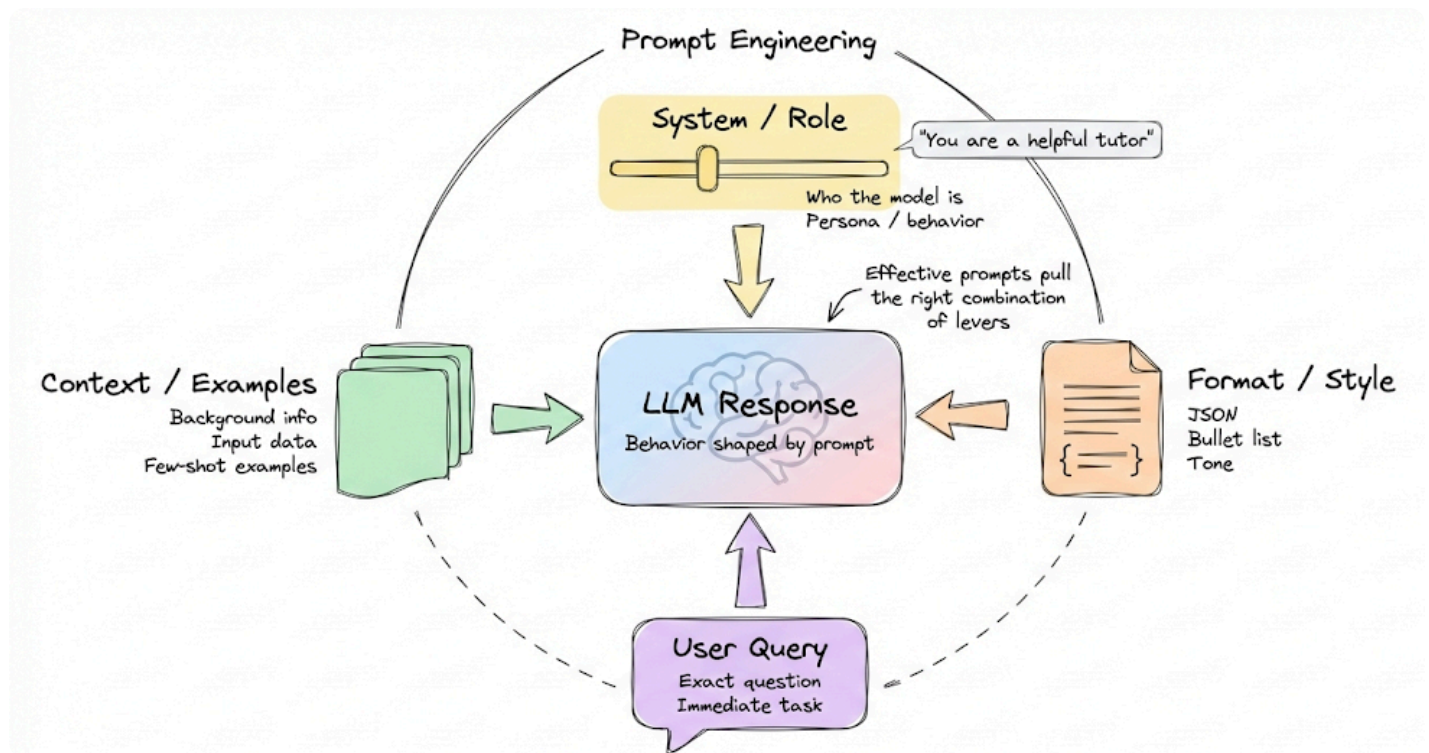
Often, they appear in system messages (e.g. “Always answer in markdown format.”) or at the end of a user prompt (“... Give the answer as a JSON.”). Because LLMs can sometimes ignore format instructions if not stated clearly, it’s good practice to be very explicit and even show an example of the format if critical.

With this we have covered the major categories of prompts. One thing to note is, these categories are not mutually exclusive, a single prompt often involves a system role definition, a user query, and maybe some examples all together.



The key takeaway is that prompt engineering gives us multiple levers: we can shape who the model pretends to be (system/role), what it should consider (context/examples), how to respond (format/style), and exactly what question to answer (user query).

Effective prompt development means pulling the right combination of these levers for the task at hand.



Next, we'll explore some prompting techniques that have been developed to push LLMs to better performance on complex tasks.

Prompting techniques

Basic prompting can be incredibly powerful, but researchers and practitioners have found techniques to push LLMs even further. We'll discuss some of these advanced prompting techniques and how they work:

Chain-of-thought (CoT) prompting

Chain-of-thought (CoT) prompting refers to a class of prompting techniques that explicitly encourage a language model to decompose a problem into intermediate reasoning steps before producing a final answer.

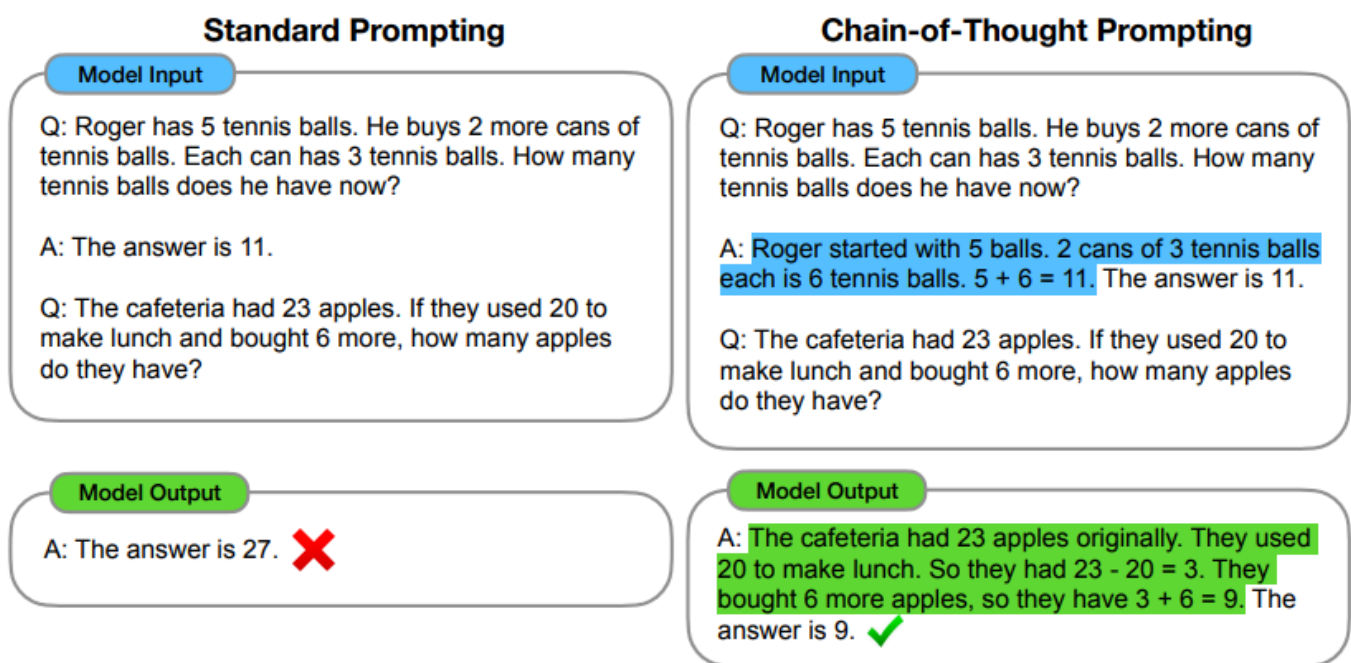
From a developer's perspective, CoT is not merely a stylistic prompt tweak; it is a mechanism for inducing structured latent computation in autoregressive models that otherwise tend to

shortcut directly to answers.



In standard prompting, an LLM is optimized to map an input directly to an output token sequence that maximizes likelihood. For tasks involving multi-step arithmetic, symbolic manipulation, or logical dependencies, this often leads to brittle behavior: the model may “jump” to an answer pattern without correctly executing intermediate steps.

CoT prompting alters this dynamic by reshaping the decoding trajectory so that intermediate reasoning tokens become part of the predicted sequence.



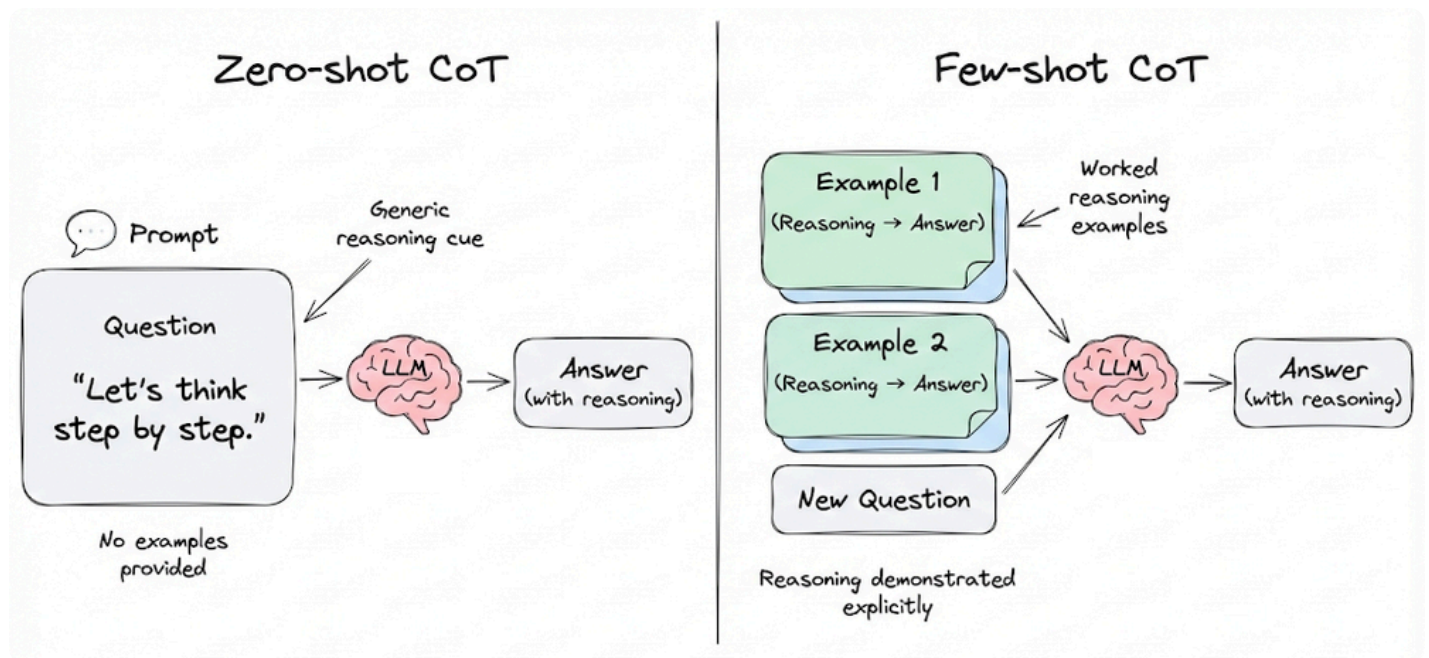
Screenshot from [Wei et al., 2022](#).

It has been empirically demonstrated that sufficiently large models exhibit a strong emergent ability to perform multi-step reasoning when prompted with explicit reasoning traces.

Crucially, this capability may not reliably appear in smaller models, highlighting that CoT is not just a prompt trick but a scale-dependent phenomenon.

From an implementation standpoint, CoT prompting typically takes one of two forms:

- Zero-shot CoT, where the developer adds a generic reasoning cue such as “Let’s think step by step.”
- Few-shot CoT, where the prompt includes one or more worked examples that explicitly show intermediate reasoning before the final answer.



Few-shot CoT is generally more robust, as it conditions the model on the structure of reasoning rather than relying on a single heuristic phrase. However, it also consumes more context window and must be carefully curated to avoid overfitting to specific patterns.





Note: CoT does not expose the model's true internal reasoning. The generated reasoning tokens are themselves predictions, not introspections. CoT works because autoregressive generation forces the model to maintain consistency across steps, effectively externalizing a computation that would otherwise remain implicit.

In practice, CoT improves performance on tasks that require:

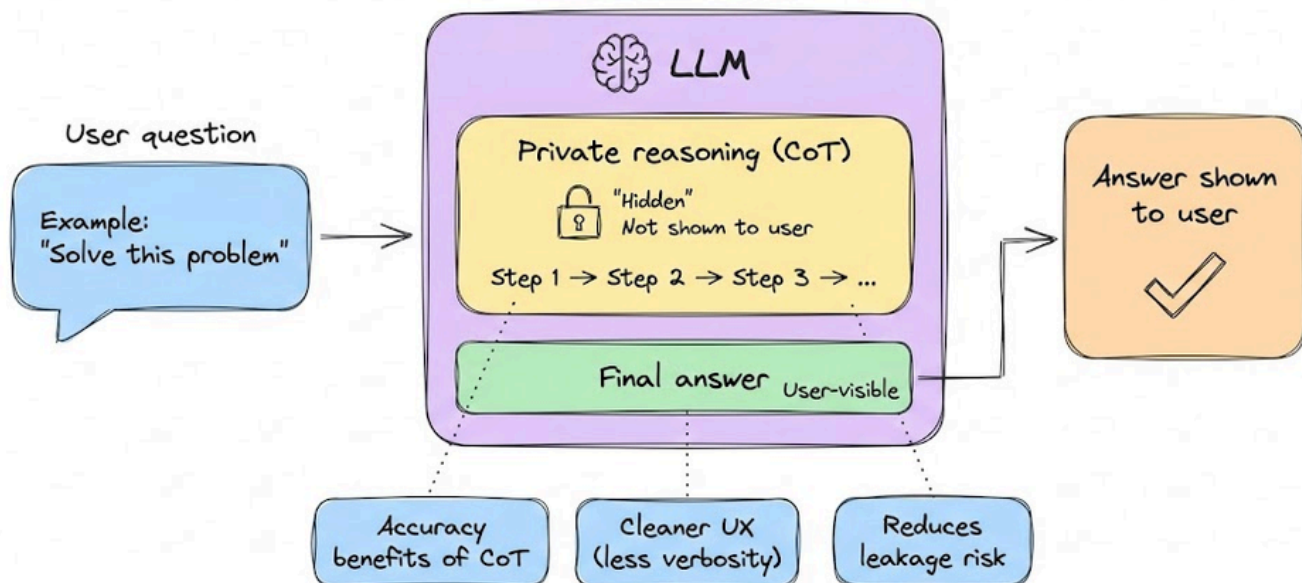
- Multi-step arithmetic or algebra
- Logical deduction and constraint satisfaction
- Commonsense reasoning with causal chains
- Program synthesis and algorithmic reasoning
- Planning-style problems in agentic workflows

Math Word Problems (free response) Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.	Math Word Problems (multiple choice) Q: How many keystrokes are needed to type the numbers from 1 to 500? Answer Choices: (a) 1156 (b) 1392 (c) 1480 (d) 1562 (e) 1788 A: There are 9 one-digit numbers from 1 to 9. There are 90 two-digit numbers from 10 to 99. There are 401 three-digit numbers from 100 to 500. $9 + 90(2) + 401(3) = 1392$. The answer is (b).	CSQA (commonsense) Q: Sammy wanted to go to where the people were. Where might he go? Options: (a) race track (b) populated areas (c) desert (d) apartment (e) roadblock A: The answer must be a place with a lot of people. Race tracks, desert, apartments, and roadblocks don't have a lot of people, but populated areas do. So the answer is (b).
StrategyQA Q: Yes or no: Would a pear sink in water? A: The density of a pear is about 0.6 g/cm^3 , which is less than water. Thus, a pear would float. So the answer is no.	Date Understanding Q: The concert was scheduled to be on 06/01/1943, but was delayed by one day to today. What is the date 10 days ago in MM/DD/YYYY? A: One day after 06/01/1943 is 06/02/1943, so today is 06/02/1943. 10 days before today is 05/23/1943. So the answer is 05/23/1943.	Sports Understanding Q: Is the following sentence plausible? "Joao Moutinho caught the screen pass in the NFC championship." A: Joao Moutinho is a soccer player. The NFC championship is part of American football, not soccer. So the answer is no.
SayCan (Instructing a robot) Human: How would you bring me something that isn't a fruit? Explanation: the user wants something to eat that isn't a fruit. An energy bar is not a fruit, so I will bring the user an energy bar. Plan: 1. find(energy bar) 2. pick(energy bar) 3. find(user) 4. put(energy bar) 5. done().	Last Letter Concatenation Q: Take the last letters of the words in "Lady Gaga" and concatenate them. A: The last letter of "Lady" is "y". The last letter of "Gaga" is "a". Concatenating them is "ya". So the answer is ya.	Coin Flip (state tracking) Q: A coin is heads up. Maybelle flips the coin. Shalonda does not flip the coin. Is the coin still heads up? A: The coin was flipped by Maybelle. So the coin was flipped 1 time, which is an odd number. The coin started heads up, so after an odd number of flips, it will be tails up. So the answer is no.

Examples of <input, chain of thought, output> triples for arithmetic, commonsense, and symbolic reasoning benchmarks. [Wei et al., 2022](#).

Empirically, even minimal zero-shot CoT often increases accuracy, but it also increases token usage, latency, and variance. For production systems, this creates a clear trade-off: performance/quality versus cost and throughput. Hence we must explicitly decide where CoT is enabled rather than applying it indiscriminately.

A common pattern is hidden or private CoT. The model is prompted to reason step-by-step internally, but only the final answer is surfaced to the user. This preserves the accuracy benefits of CoT while avoiding verbosity, UX clutter, and potential leakage of sensitive intermediate reasoning.



To illustrate the behavioral difference, consider a simple arithmetic word problem. Without CoT, the model may directly output an answer token. With CoT prompting, the model is far more likely to explicitly compute totals, apply transformations, and then conclude. While this example is trivial, the effect compounds dramatically for longer dependency chains.

Standard Prompting	Chain-of-Thought Prompting
Model Input Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: The answer is 11. Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?	Model Input Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11. Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?
Model Output A: The answer is 27. ❌	Model Output A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Screenshot from [Wei et al., 2022.](#)

Now that we understand benefits of CoT, it is also important to understand the failure modes of CoT. Because reasoning tokens are generated autoregressively, the model can still hallucinate incorrect steps that look plausible. Longer chains increase surface-level coherence but do not guarantee correctness. This has motivated extensions such as self-consistency which we'll discuss ahead, and tool-augmented CoT, where intermediate steps invoke calculators, solvers, or retrieval systems.

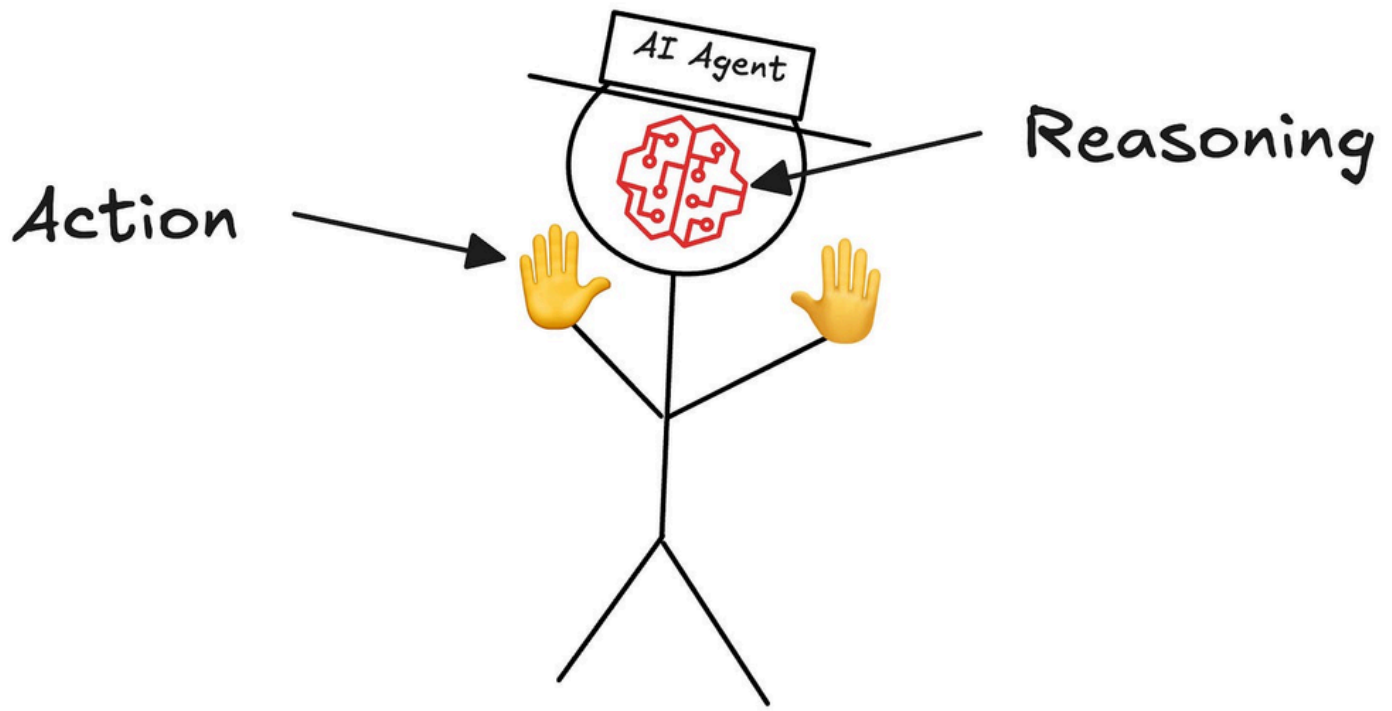


From a system's design perspective, CoT should be viewed as one component in a broader reasoning stack rather than a silver bullet. It works best when combined with verification, tool use, or constrained decoding.

In summary, chain-of-thought prompting is best understood not as “making the model explain itself,” but as a controllable way to elicit multi-step computation. When applied deliberately, it is one of the most effective tools available for improving reasoning performance in LLM-powered systems.

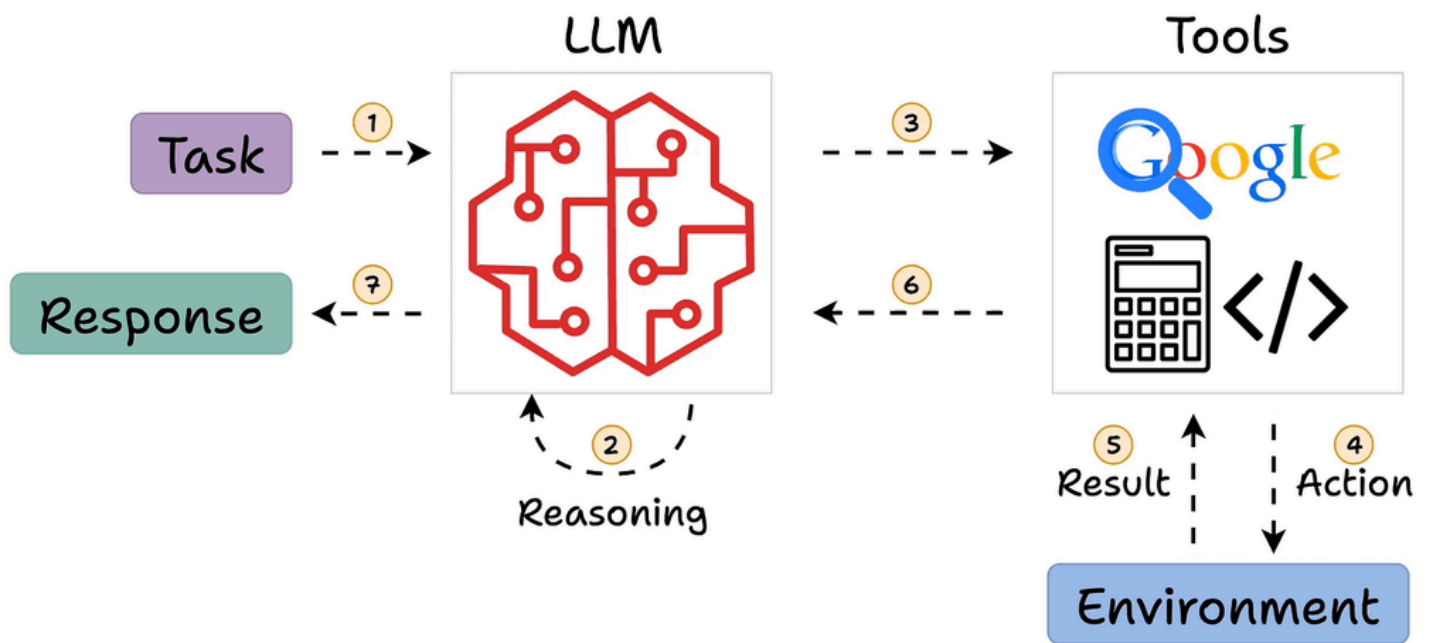
ReAct prompting

ReAct (Reason + Act) is a prompting and control framework that tightly couples explicit reasoning traces with environment-interacting actions.



Unlike pure chain-of-thought prompting, which externalizes reasoning but still assumes the model answers in a single forward pass, ReAct turns the model into a decision-making loop that can iteratively reason, act, observe results, and update its plan.

From a technical perspective, ReAct is best understood as an LLM-driven policy over tools, where natural-language reasoning tokens guide action selection, and observations from the environment condition subsequent reasoning. The prompt format is simply the interface contract that enforces this loop.

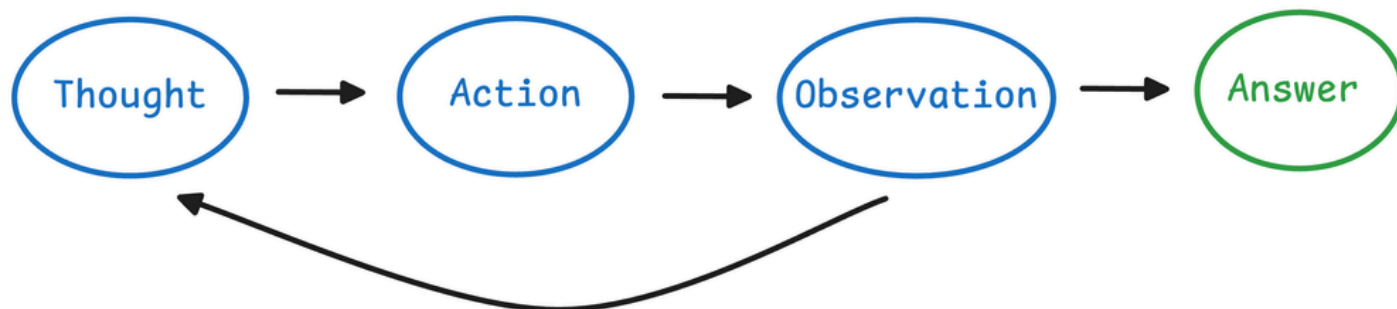


The core insight is that reasoning and acting are complementary. When a model reasons without tools, it is forced to guess or hallucinate missing information. When it invokes tools without making its reasoning explicit, the resulting behavior becomes opaque, hard to debug, and brittle. ReAct addresses this by interleaving reasoning and tool use in the same autoregressive sequence.

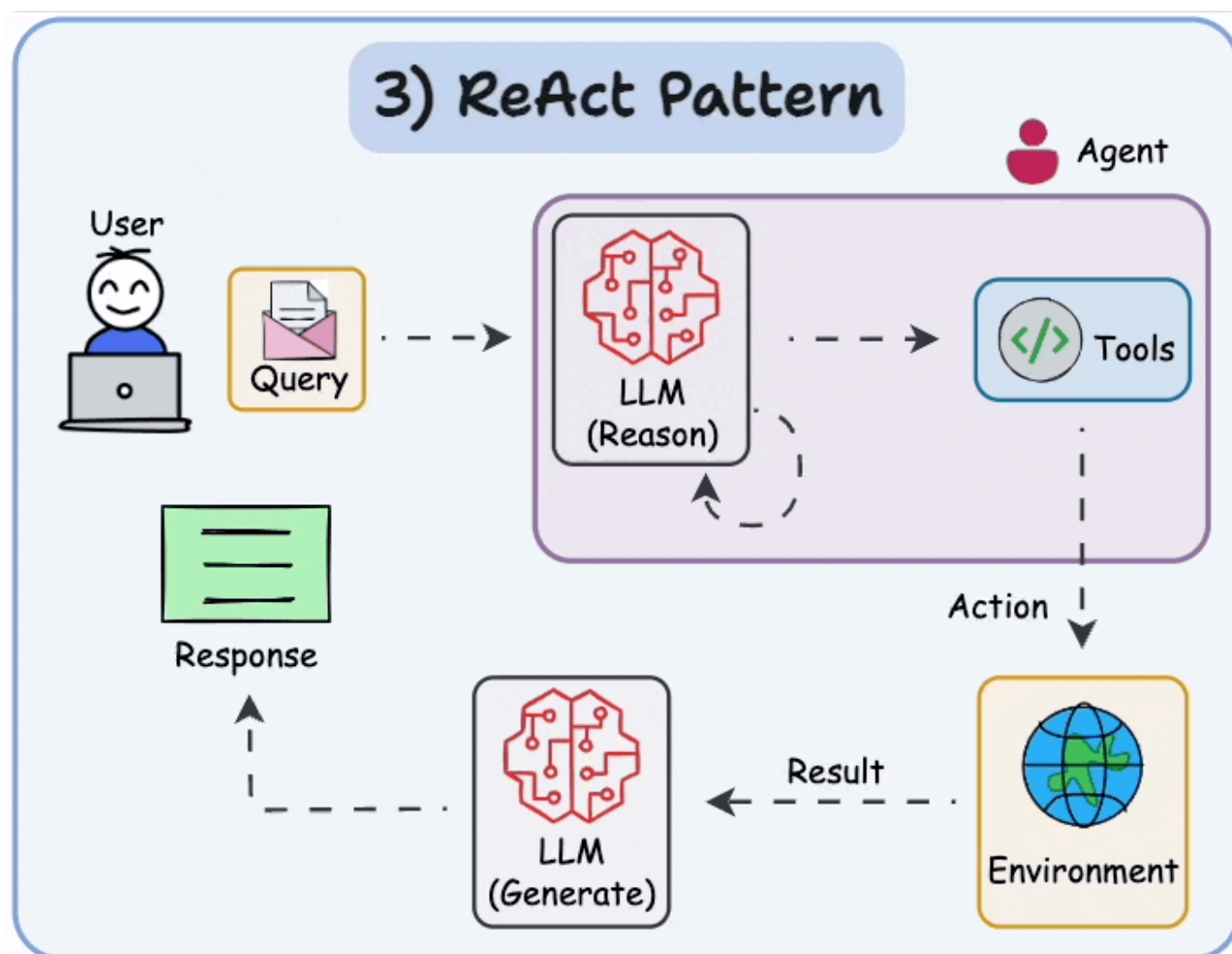
A typical ReAct trajectory alternates through three conceptual states:

- **Thought:** the model's explicit reasoning about what it knows, what it lacks, and what to do next.
- **Action:** a structured instruction emitted by the model that maps to a tool invocation.
- **Observation:** external state returned by the system after executing the action.

ReAct Pattern



Crucially, the model does not execute actions itself. The LLM only emits text. The surrounding runtime (code) parses the action, executes it, and appends the observation back into the prompt. This makes ReAct a prompt-driven control loop, not an autonomous executor.



From an implementation standpoint, ReAct requires three pieces of scaffolding:

- A strict output grammar: The model must emit actions in a machine-parsable format. Free-form text is unacceptable here. Most implementations enforce patterns like `Action: ToolName[arguments]` to simplify parsing and prevent ambiguity.

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: `Search[Apple Remote]`

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: `Search[Front Row]`

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: `Search[Front Row (software)]`

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: `Finish[keyboard function keys]`



Screenshot from [Yao et al., 2023](#) demonstrating output grammar.

- An execution harness: A loop that:
 - Sends the current prompt to the model.
 - Detects whether the model produced an action or a final answer.

- Executes the action safely.
- Injects the observation back into the context.
- Repeats until termination.

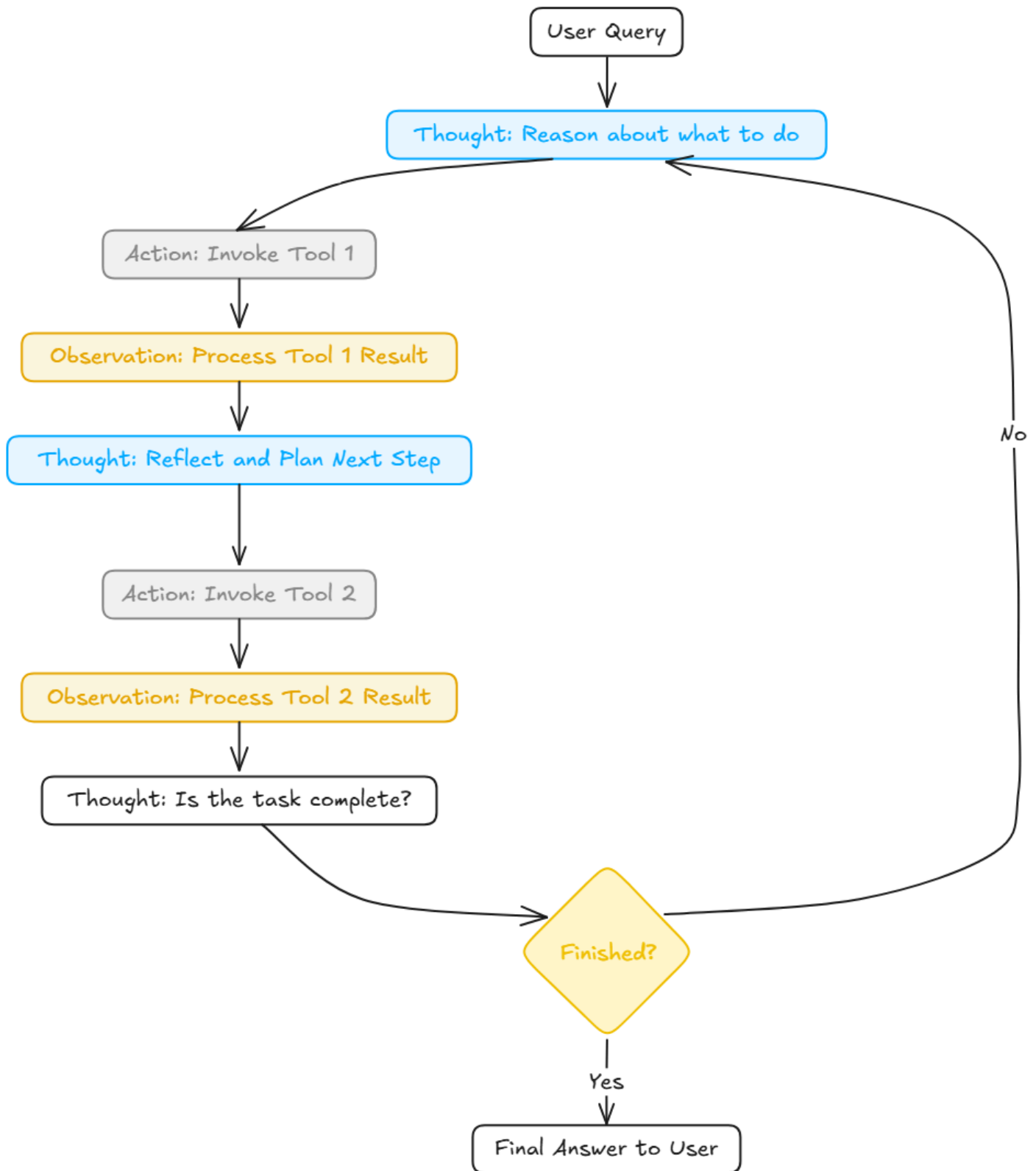
Termination conditions: The system must decide when to stop. This is usually done via a `Finish` or `Answer` action, max-step limits, or confidence thresholds. Without explicit stopping logic, ReAct agents can loop indefinitely.

```
Thought 4: Front Row (software) is controlled by an Apple  
Remote or the keyboard function keys. So the answer is  
keyboard function keys.  
Act 4: Finish[keyboard function keys]
```

Screenshot from [Yao et al., 2023](#) demonstrating "Finish" action.

Here's the entire loop in form of a flowchart:

ReAct pattern



One of the most important technical distinctions for developers is that ReAct reasoning tokens are not optional narration. They function as state carriers. The reasoning text helps the model

maintain coherence across steps, remember past observations, and justify future actions.

(2b) ReAct (Reason + Act)

Act 1: **Think**[First I need to find a pepper shaker...more likely to appear in cabinets (1-6), countertops (1-3), ...]

Act 2: Go to cabinet 1

Obs 2: On the cabinet 1, you see a vase 2.

(Here the agent go to cabinet 1, then cabinet 2, 3, then countertop 1 and 2)

Act 6: Go to countertop 3

Obs 6: On the countertop 3, you see a apple 1, a bread 1, a pepper shaker 1, and a vase 3.

Act 7: Take pepper shaker 1 from countertop 3

Obs 7: You pick up the pepper shaker 1 from the countertop 3.

Act 8: **Think**[Now I find a pepper shaker 1. Next, I need to put it in/on drawer 1.]

Act 9: Go to drawer 1

Obs 9: Drawer 1 is closed.

Act 10: Open drawer 1

Obs 10: You open Drawer 1 ...

Act 11: Put pepper shaker 1 in/on drawer 1

Obs 11: You put pepper shaker 1 in/on the drawer 1.



Screenshot from [Yao et al., 2023](#).

That said, ReAct reasoning is still generated, not guaranteed to be correct. This introduces a subtle but critical trade-off: ReAct improves tool grounding and factual accuracy, but it also increases the surface area for failure.

A single incorrect reasoning step can cascade into repeated bad actions. For this reason, production systems often combine ReAct with guardrails such as:

- Tool-level permissioning (like in MCP)

Let's discuss the QuboAI app further. What did I tell you about it? Use MCP tool

Waiting for approval...

Stop ⬆ ⌘ ⌫

✓ Calling MCP tool `search_facts`

```
{  
  "query": "QuboAI app"  
}
```

Approval

Cancel ⌘ ⌫

Run tool ⌘ ↵

- Argument validation and sanitization
- Observation truncation or summarization
- Secondary verification models

In practice, ReAct excels at tasks that require information gathering or environment interaction, such as:

- Question answering with retrieval
- Multi-hop reasoning over documents or APIs
- Data analysis with calculators

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: `Search[Apple Remote]`

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: `Search[Front Row]`

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: `Search[Front Row (software)]`

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: `Finish[keyboard function keys]`



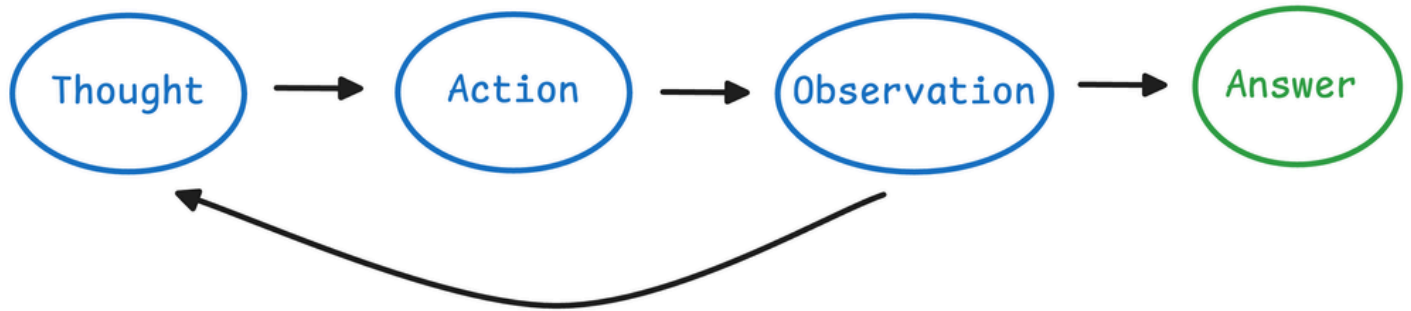
ReAct on HotpotQA. [Yao et al., 2023](#).

Note that ReAct performs poorly on tasks that are fully solvable in a single forward pass, where the overhead of tool calls outweighs any benefit.

An important architectural insight from the ReAct paper is that the framework does not require fine-tuning. The behavior emerges from prompt structure alone, assuming the model is sufficiently capable. This made ReAct especially influential, as it demonstrated that agent-like behavior can be induced purely through inference-time control.

Modern agent frameworks build directly on this idea, though many abstract away the explicit labels. Under the hood, however, most still implement some variant of the same loop: reason → decide → act → observe → repeat.

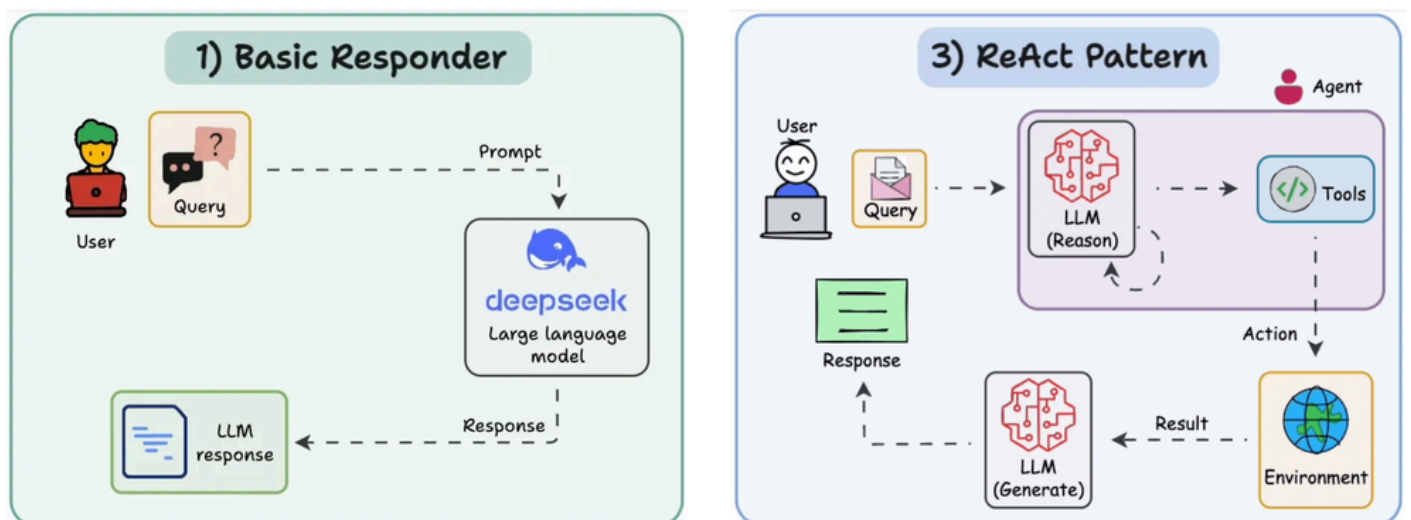
ReAct Pattern



From a system standpoint, it is important to recognize that ReAct is not merely a “prompting trick.” It is a programming model, where the LLM acts as a high-level planner and controller, and the host system provides deterministic execution and state updates. The prompt is effectively the agent’s policy definition.

In summary, ReAct shifts LLM usage from static input-output mapping to interactive control. It operationalizes reasoning by tying it directly to actions and observations, making it one of the foundational patterns for modern LLM agents.

When designed carefully, it enables systems that are more grounded, more capable, and more extensible than simple prompting alone but at the cost of higher system complexity and stricter engineering discipline.

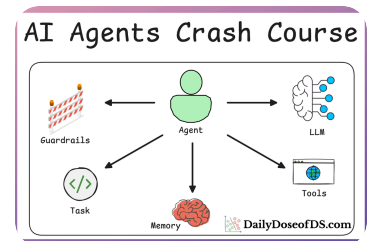


If you want to learn more about ReAct we recommend reading the below linked article from our AI agents course:

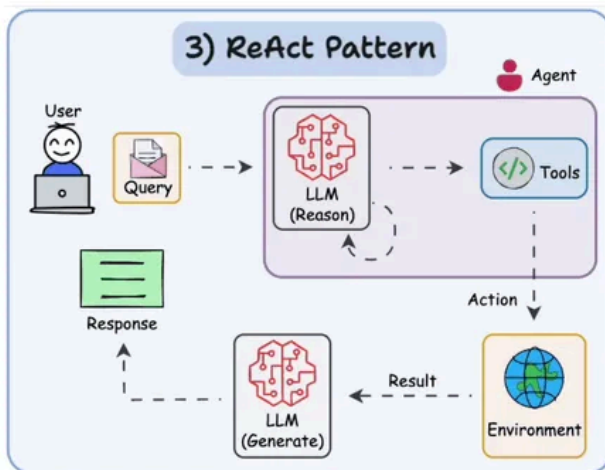
Implementing ReAct Agentic Pattern From Scratch

AI Agents Crash Course—Part 10 (with implementation).

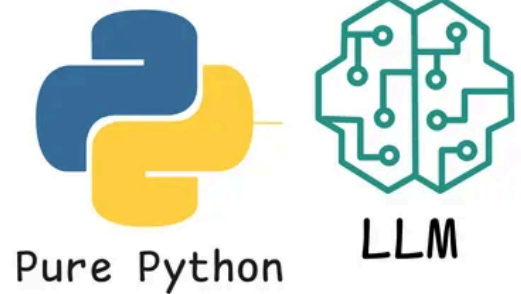
 Daily Dose of Data Science • Avi Chawla



In the article, you will understand the entire process of building a ReAct agent from scratch using only Python and an LLM:



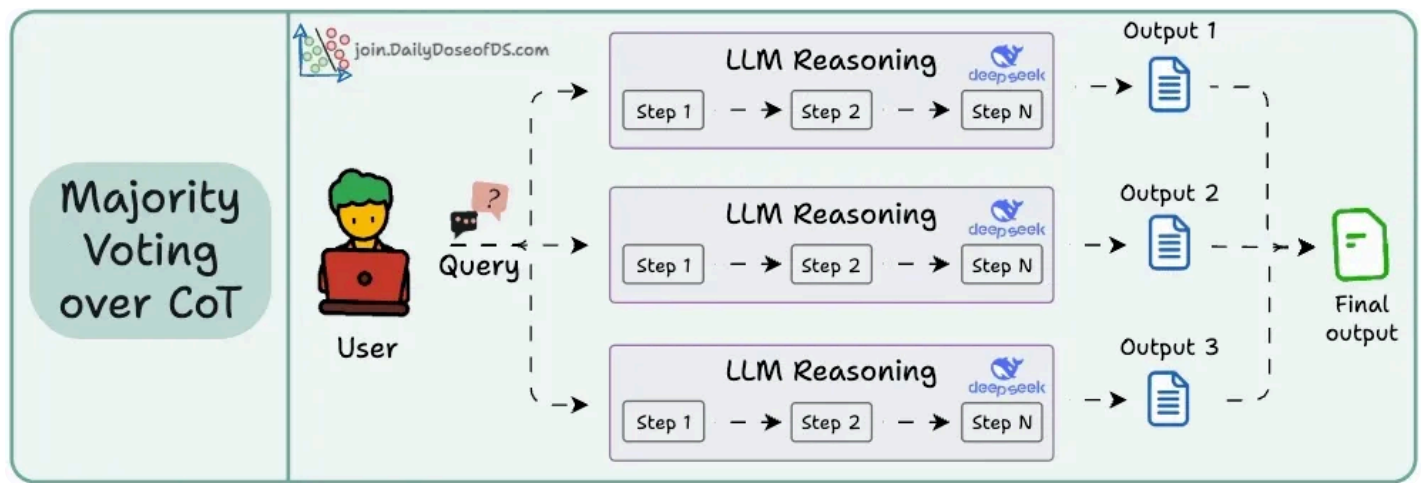
Implemented
with



Self-consistency

Self-consistency is a technique aimed at improving the reliability of chain-of-thought reasoning. Instead of trusting a single reasoning path, we prompt the model to generate multiple independent solutions and then aggregate their results.

In practical terms, you run the model N times for the same question (each time it might produce a different chain-of-thought due to randomness in sampling), and then you see which final answer is most common among those attempts (majority voting).



The assumption is that if one answer is correct, many reasoning paths will converge to it, whereas incorrect answers will be more randomly distributed.

For example, you could sample 5 chains-of-thought for a tricky math problem and find that 4 out of 5 conclude the answer is 42, while one says 41. By majority vote, you choose 42 as the final answer, increasing the likelihood of correctness. This is essentially an ensemble method at the prompt level.

👉 Implementing self-consistency requires using a stochastic decoding method (like sampling with temperature) to get diverse solutions, and then either majority vote or some confidence scoring to pick the answer.

It trades additional compute cost for higher reliability. In an LLMops context, you'd use this when correctness is paramount and you can afford the extra calls. For instance, critical reasoning tasks might justify running 5 model instances in parallel and voting.

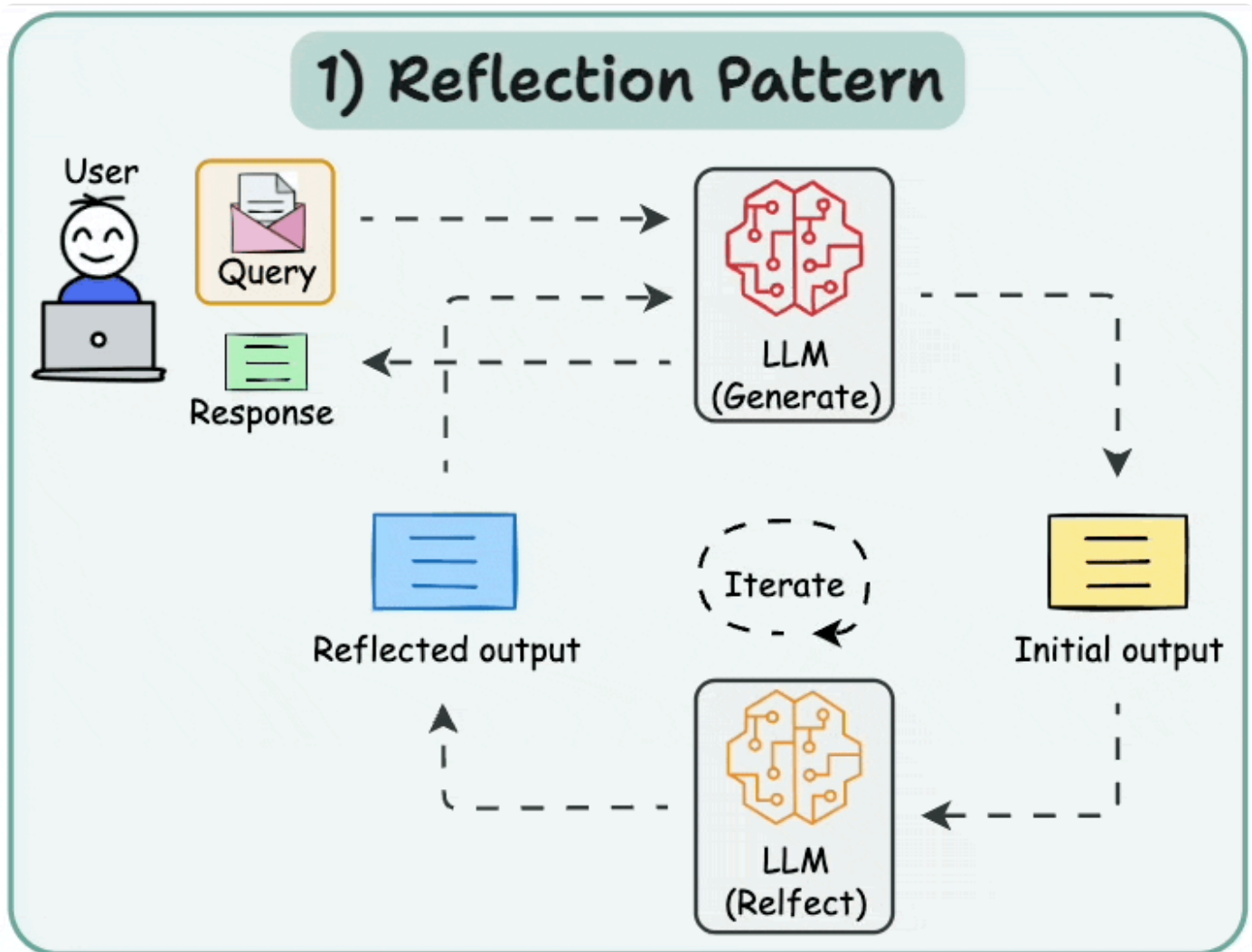
The advantage is you don't have to change the model or fine-tune it; you're simply leveraging its own variability to your benefit.

👉 One must be cautious that all the sampled outputs aren't identical (ensuring enough temperature or randomness), otherwise it's not useful. Also, if the model has a consistent bias, self-consistency won't fix it (all answers could be consistently wrong).

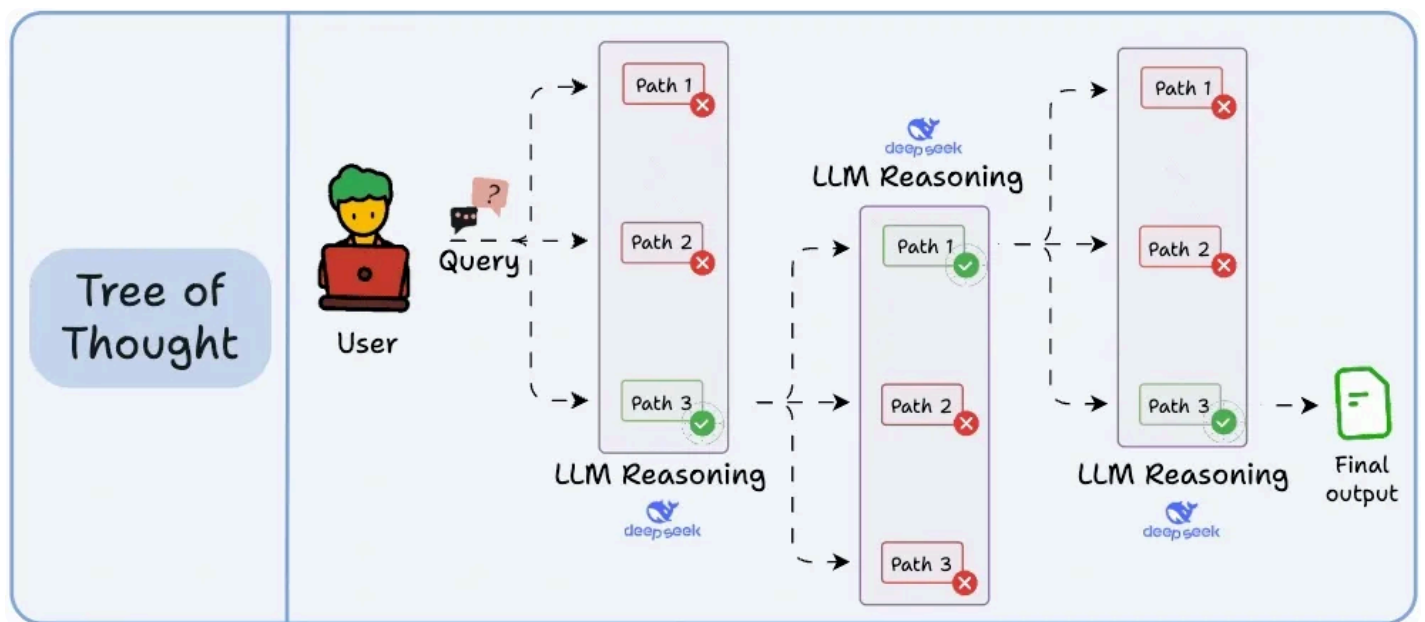
Other techniques

There are numerous other prompting approaches present. To name a few:

- Reflection prompts the model to reflect on its answer and possibly correct itself. This could mean after getting an answer, you ask the model “Are you sure? Evaluate the answer and correct if needed.” and let it produce a refined answer. This can catch mistakes by having the model critique its own output.



- Another idea is expert/novice prompting, where you prompt the model to act as an expert and also produce an explanation a novice could understand, which can improve clarity.
- Tree-of-thought is an advanced method where the model explores a decision tree of reasoning paths instead of one linear chain. This can be useful for complex planning problems.



- There is also automatic prompt engineering (APE), a process of using other AI models or algorithms to automatically create, test, and refine prompts, replacing manual trial-and-error to find the most effective instructions. In its simplest form, you can ask a model to generate a prompt for your needs as shown in the screenshot below.

Help me write a very concise prompt for an AI application that creates science quizzes for high school grade students, with 10 questions of variable difficulty.

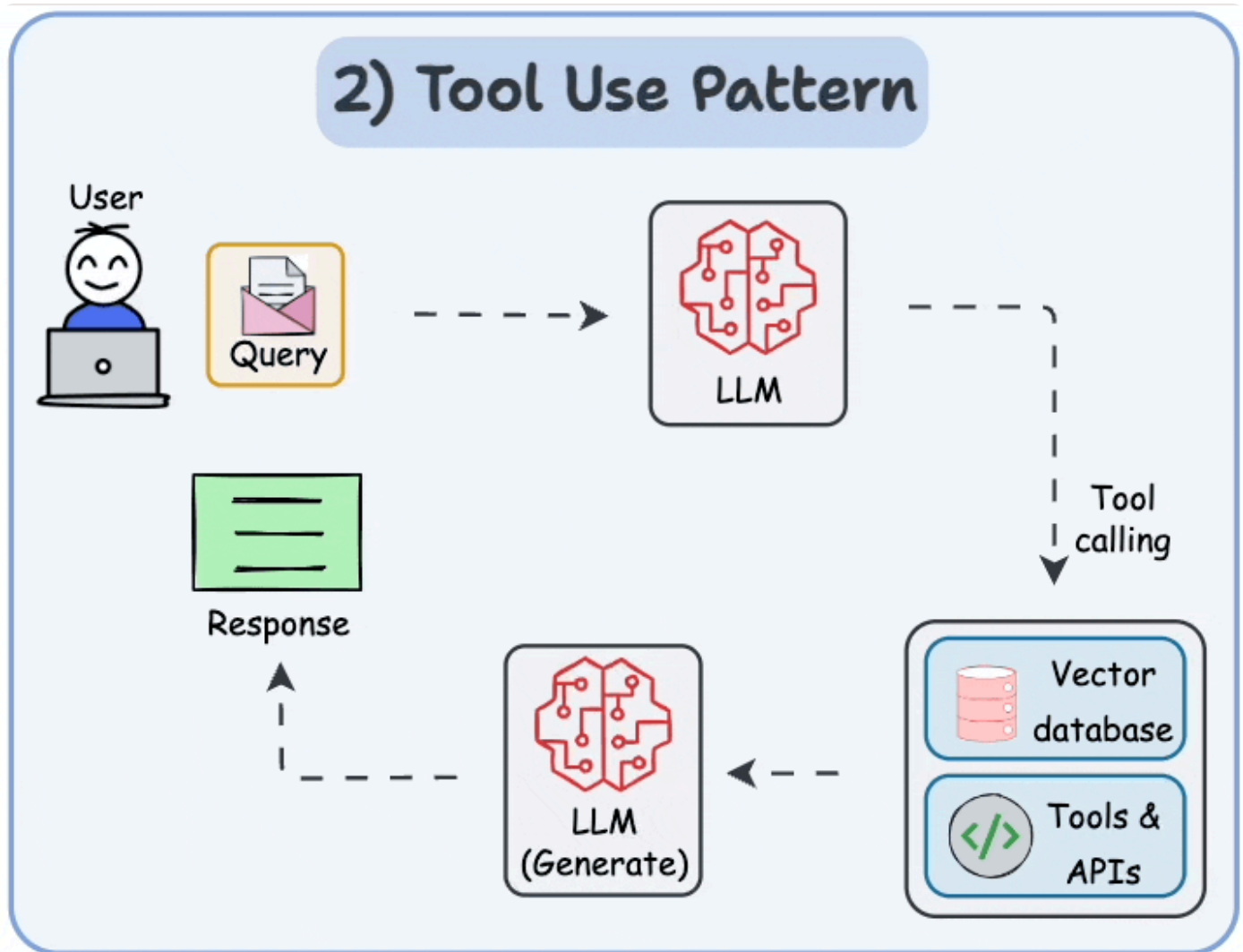
Here's a concise prompt for your quiz generator:

Generate a 10-question science quiz for high school students. Include a mix of difficulty levels (easy, medium, hard). Cover topics appropriate for grades 9-12. For each question, provide: the question text, 4 multiple choice options (A-D), the correct answer, and a brief explanation. Ensure questions test understanding, not just memorization.

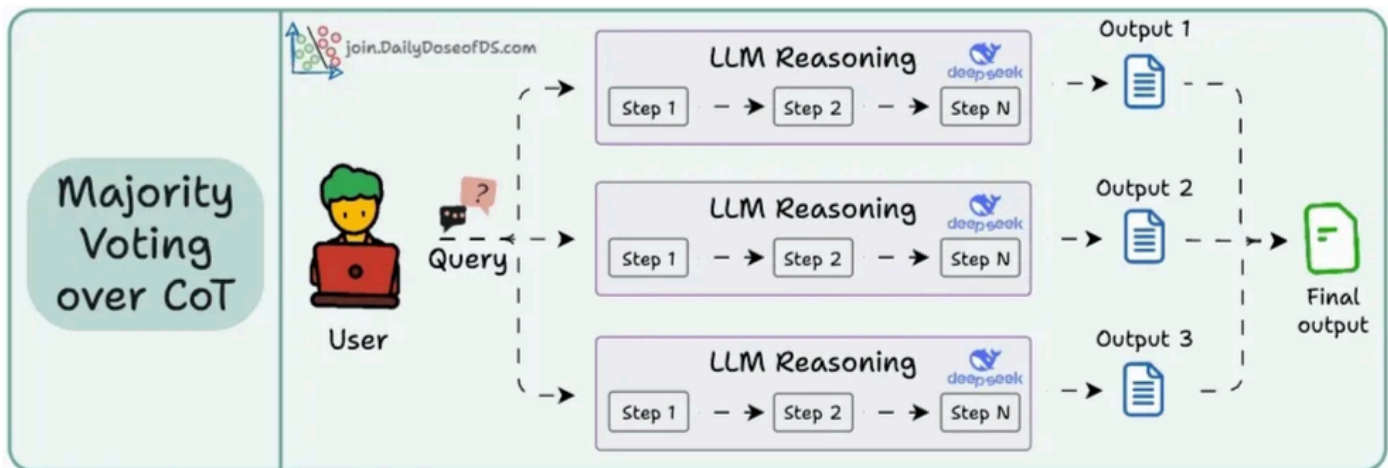
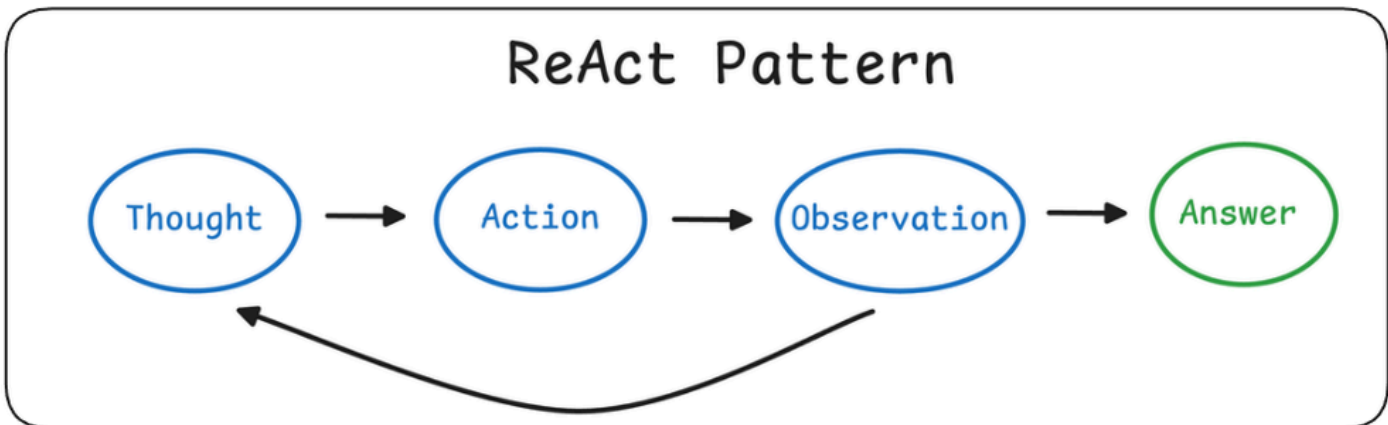
A bare minimum example of APE

👉 You can also ask AI models to critique and improve your prompts or generate in-context examples.

Many advanced techniques combine prompting with external processes (tool use, etc.), blurring the line between "pure prompt engineering" and "algorithm design" and context analysis.



The takeaway is that prompt engineering has a growing toolkit of patterns to systematically reduce errors and improve outputs: guiding the model to reason (CoT), to interact with its environment (ReAct), to generate diverse solutions (self-consistency), and so on.



We've covered some core ones here, as these are widely applicable. In practice, you might mix techniques, for instance, a ReAct-based agent that uses chain-of-thought in its reasoning steps and employs self-consistency when answering critical questions.

With this, now we have completed the theoretical portion of this chapter. In the next section, we will explore some of these concepts through a hands-on example.

Hands-on: prompt types and CoT implementation

This hands-on section examines system prompts, user prompts, and formatting prompts in combination with a zero-shot chain-of-thought pattern to implement a basic math word problem solver.

Project setup

This project's code is meant to be run using Google Colab. We recommend uploading the `.ipynb` notebook in Colab, and running it from there using CPU-based runtime.

Download the notebook here:

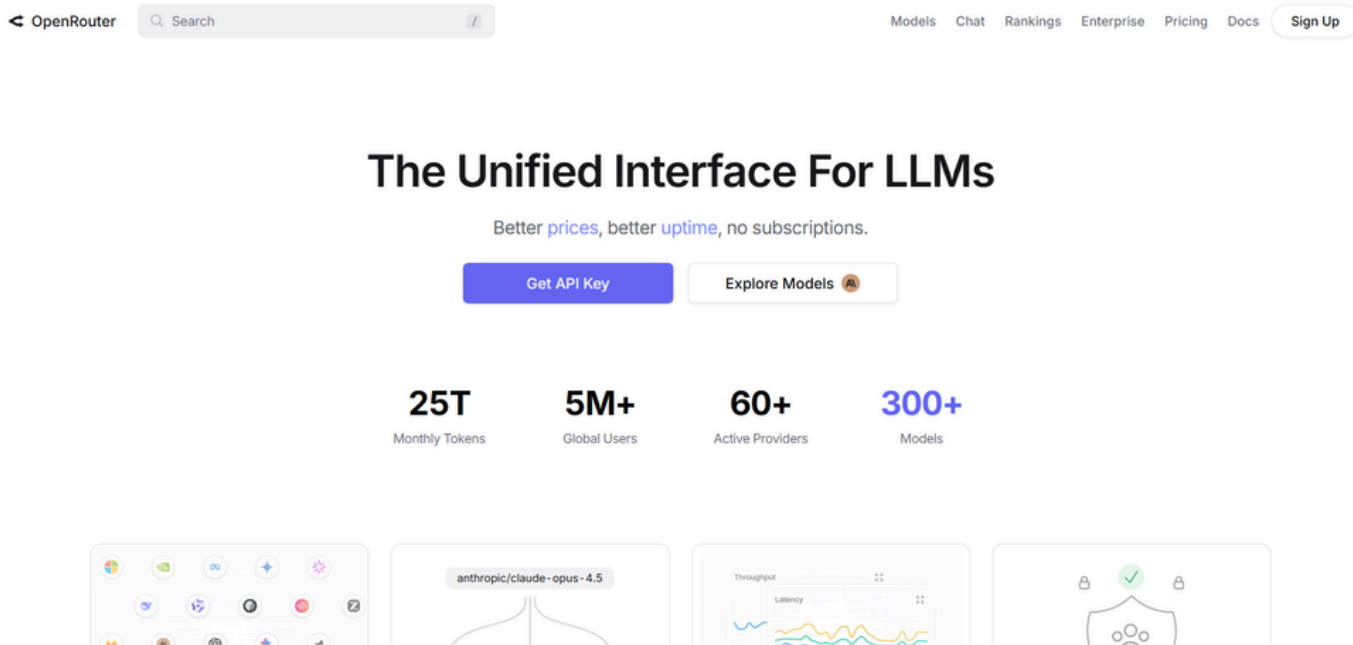
cot_demo



cot_demo.ipynb • 19 KB

Important:

Sign up and obtain your API key from <https://openrouter.ai/>.



By default, we use `gpt-4o-mini`. However, OpenRouter also provides several free models that you can access using the same API key.

For example: `openrouter/meta-llama/llama-3.3-70b-instruct:free`, `openrouter/meta-llama/llama-3.1-405b-instruct:free`.

Alternative option: You can also use Groq instead of OpenRouter. Groq offers limited free inference for certain models as well. With LiteLLM, the usage format is: `groq/<model-name>`.

You can refer to the official LiteLLM Groq provider guide

here: <https://docs.litellm.ai/docs/providers/groq>

Final note: While free models are supported, we recommend using `gpt-4o-mini`. Free models are often under heavy demand, which can lead to frequent rate limits or API failures. A small top-up on OpenRouter is usually worth it.



Or, if you have an OpenAI key, then you can switch providers entirely and use OpenAI directly with LiteLLM, for example: `openai/gpt-4o-mini` instead of `openrouter/openai/gpt-4o-mini`, and use your OpenAI API key instead of OpenRouter key.

Implementing a basic math word problem solver

Now that we have our setup done, let's start with our actual demo.

What our script is doing, end-to-end, is:

Ask an LLM to solve a math word problem with a reasoning section, strip that reasoning out before showing the user, and save the reasoning to a local text file, for record.

It's basically a tiny “math-solver pipeline” with three outputs:

- raw model response (contains both reasoning + final solution),
- cleaned response (final solution only),

- extracted reasoning (saved to `math_cot_reasoning.txt` with a timestamp).

Now let's walk through the code step-by-step. Firstly we define our system prompt and output format:



```
... # imports and config (check notebook)
```

```
MODEL = "openrouter/openai/gpt-4o-mini"
```

```
system_prompt = """
```

```
You are an expert mathematics tutor and problem solver.
```

**system
prompt**

```
You must produce TWO parts in this exact order:
```

```
1) A step-by-step solution inside <thought>...</thought> tags
```

```
showing all your reasoning.
```

```
2) The final solution in STRICT Markdown format exactly as specified.
```

```
Do not add anything else outside these two parts.
```

```
"""
```

```
SOLUTION_FORMAT_SPEC = """
```

```
## SOLUTION_START
```

```
### PROBLEM_UNDERSTANDING
```

```
<Brief restatement of what the problem is asking>
```

**output
format**

```
### APPROACH
```

```
<The method or strategy used to solve the problem>
```

```
### SOLUTION_STEPS
```

```
1. <Step 1>
```

```
2. <Step 2>
```

```
3. <Step 3>
```

```
...
```

```
### FINAL_ANSWER
```

```
<The final numerical answer with appropriate units>
```

```
### VERIFICATION
```

```
<Quick check that the answer makes sense>
```

```
## SOLUTION_END
```

```
""".strip()
```

In the above code we use `re` for regex, and configure `litellm` to call a chat model through a unified interface, and as mentioned earlier, the model in use is: `"openrouter/openai/gpt-4o-mini"`.

Then we go ahead and define two prompt chunks:

1. `system_prompt`
2. `SOLUTION_FORMAT_SPEC`

System prompt is the “global behavior contract” for the model. We are telling it to produce TWO parts:

- step-by-step reasoning inside `<thought>...</thought>`
- final solution in a strict Markdown template

The “do not add anything else outside these two parts” enforces a strict condition that the model should not generate anything beyond the specified two parts.

Thus, overall, our system prompt is enforcing a protocol.

Coming to `SOLUTION_FORMAT_SPEC`, this is the exact Markdown structure we want the final answer to follow:

- `## SOLUTION_START`
- sections like `### PROBLEM_UNDERSTANDING`, `### APPROACH`, etc.
- `## SOLUTION_END`



This is useful in scenarios where we might want downstream parsing later (for example, splitting the response into labeled parts consistently).

By this point in our code, we have understood system prompts and formatting prompts. Now, let us go ahead and take a look at our two helper utilities:

The `extract_and_strip_thoughts(...)` function/utility does two jobs:

- Extract all reasoning blocks using `regex` and combine them together as `thought`.
- Remove the reasoning from the visible output `cleaned`

Coming to our next utility, as suggested by its name, it appends the reasoning to a text file.

This is what we see getting saved when run our complete script:

```

1
2 =====
3 [2026-01-23 11:31:24]
4 1. **Identify what's given and what's being asked**:
5   - Given: Sarah has 3 boxes of cookies, and each box contains 24 cookies.
6   - Total friends: 8.
7   - We need to find out how many cookies each friend will receive and if there will be any cookies left over.
8
9 2. **Plan your approach**:
10  - First, calculate the total number of cookies Sarah has by multiplying the number of boxes by the number of cookies in
11  - Then, divide the total number of cookies by the number of friends to find out how many cookies each friend gets.
12  - Finally, check if there are any cookies left over by calculating the remainder of the division.
13
14 3. **Work through each calculation with explanations**:
15  - Total cookies = Number of boxes  $\times$  Cookies per box = 3  $\times$  24 = 72 cookies.
16  - To find out how many cookies each friend receives, we divide the total cookies by the number of friends:
17  - Cookies per friend = Total cookies  $\div$  Number of friends = 72  $\div$  8 = 9 cookies.
18  - To check for leftovers, we calculate the remainder:
19  - Remainder = Total cookies  $\%$  Number of friends = 72  $\%$  8 = 0 cookies.
20
21 4. **Check your work**:
22  - We calculated the total cookies correctly as 72.
23  - Dividing 72 by 8 gives us 9, which is correct.
24  - The remainder is 0, indicating there are no cookies left over.
25

```

In the screenshot, you can see the timestamp line and then the extracted CoT reasoning content.

This block is not what gets printed; it's what gets saved to disk.

Now finally let us come to our main solver function: `solve_math_problem(problem: str)`:

This constructs a `user_prompt` (template) that contains:

- Instructions (repeat the two tasks)
- The strict Markdown template
- The actual problem text inside triple quotes

Then it calls `completion(...)`, with `temperature=0.2`. This takes the model more towards deterministic side and less “creative”, which is exactly what we want to ensure our defined structures are followed.

Then we extract the model response and do post-processing with:

- `extract_and_strip_thoughts`
- `save_thought_to_file`
- `return final_output`

So the CoT reasoning gets saved and the function returns ONLY the clean Markdown solution.

With this we have understood this snippet also.

Now that our entire code logic is defined, let us go ahead and define the `__main__` block with a sample input:

We define `sample_problem` (Sarah + cookies) and then:

- `solution = solve_math_problem(sample_problem)`
- `print(solution)`
- print a final line: `(Chain-of-thought reasoning saved to ./math_cot_reasoning.txt)`

This is the output that we see on running the code:

```
## SOLUTION_START

### PROBLEM_UNDERSTANDING
Sarah has 3 boxes of cookies, each containing 24 cookies, and wants to distribute them equally among 8 friends. We need to determine how many cookies each friend receives.

### APPROACH
Calculate the total number of cookies, divide that total by the number of friends to find out how many cookies each friend gets, and check for any remainder to see if there are any leftovers.

### SOLUTION_STEPS
1. Calculate the total number of cookies: 3 boxes × 24 cookies/box = 72 cookies.
2. Divide the total cookies by the number of friends: 72 cookies ÷ 8 friends = 9 cookies per friend.
3. Check for leftovers: 72 cookies % 8 friends = 0 cookies left over.

### FINAL_ANSWER
Each friend will receive 9 cookies, and there will be no cookies left over.

### VERIFICATION
The calculations confirm that 72 cookies divided by 8 friends results in 9 cookies each, with no remainder, which makes sense given the total number of cookies.

## SOLUTION_END

(Chain-of-thought reasoning saved to ./math_cot_reasoning.txt)
```

- It starts at `## SOLUTION_START`

- It contains the filled sections (Problem understanding, approach, steps, final answer, verification)
- It ends at `## SOLUTION_END`
- And below that, we see the note about saving reasoning to the text file.

Notice how in the output, there are no `<thought>...</thought>` tags. That's because `extract_and_strip_thoughts()` removed them before printing.



The exact outputs and reasoning file contents will vary from those shown here; however, the overall understanding should remain consistent and unaffected.

With this we complete our experiment. As a follow-up self-learning exercise, we encourage readers to experiment with different input problems (`sample_problem`) to develop a deeper and more concrete understanding.

Also, upon further experimentation with various inputs, if you notice that the output at times fail to adhere to the defined structure and does not follow the protocols, harden and emphasize the rules more. Typical upgrades are:

- enforce the schema with examples for the final output,
- retry with a stricter system prompt when the format check fails,
- few-shot CoT instead of zero-shot,
- validate the presence of elements given in the format like: `## SOLUTION_START` and `## SOLUTION_END` before accepting the result.



Many a times structure might also fail if you are using a different model, since every model has its own learned distributions, and depending on that you might need to make a few tweaks here-n-there.

Alright now, with this, we conclude our discussion for this chapter. In the subsequent chapters, we will continue to build on the core ideas of LLMOps, and explore more intricate concepts and

their implementations wherever applicable.

Conclusion

With this chapter, we transitioned from understanding how large language models work to learning how we can intentionally shape that generation in real-world systems.

We began by reframing prompts not as casual instructions, but as engineered control surfaces. Prompting was treated as a form of soft programming, where behavior emerges probabilistically from how we structure instructions.

We then established a clear distinction between prompt engineering and context engineering, emphasizing that prompts are one way of shaping context, but not the only one. By grounding prompt engineering as a subset of a broader context pipeline, we placed it correctly within modern LLMOps architectures.

From there, we explored the fundamentals of prompting: zero-shot and few-shot in-context learning, why examples work, and how prompt length, clarity, and structure directly influence model behavior, cost, and latency.

This led naturally into a systematic prompt development workflow, showing that effective prompting is not guesswork, but an iterative engineering discipline involving specification, testing, failure analysis, refinement, and versioning.

We then examined the major prompt types used in production systems and rather than treating these as isolated categories, we emphasized how real prompts combine multiple roles and levers to guide model behavior reliably.

Moving ahead, we explored advanced prompting techniques that push models beyond simple input-output mapping: chain-of-thought, ReAct, and self-consistency.

Finally, we explored system prompts, user prompts, and formatting prompts in combination with zero-shot chain-of-thought pattern through a hands-on example.

At this point, we have answered several critical questions about prompt engineering:

- What does it mean to “program” a language model without changing its weights?
- How do examples, reasoning traces, and structure influence model reliability?
- When should we use techniques like CoT, ReAct, or self-consistency; and when should we avoid them?

Key takeaways:

- Prompts are control mechanisms, not just instructions.
- Prompt engineering is an iterative, testable, versioned engineering process.
- Few-shot learning works by conditioning behavior, but consumes context and cost.
- Advanced prompting techniques trade simplicity for reliability and capability.

With this chapter, we have completed our exploration of the fundamentals of prompt engineering. In the next chapter, we will deep dive into topics like prompt versioning, role prompting, and more.

After prompt and context engineering, we’ll continue the journey with:

- Fine-tuning and model adaptation techniques
- Evaluations
- Inference optimization
- Real-world case studies from industry of MLOps/LLMOps

The aim, as ever, is to develop a system-level perspective and to equip you with an adaptable engineering framework for building AI systems that are robust and maintainable.

As always, thanks for reading!

Any questions?

Feel free to post them in the comments.

Or

If you wish to connect privately, feel free to initiate a chat here:



A daily column with insights, observations, tutorials and best practices on python and data science. Read by industry professionals at big tech, startups, and engineering students.

chat icon



Menu

Connect via chat

Contact
Published on Jan 25, 2026
FAQ

Comments

Share

Previous — LLMOps

< Building Blocks of LLMs: Decoding, Generation Parameters, and the LLM Application Lifecycle

Next — LLMOps

Context Engineering: Prompt Management, Defense, and Control >